



Detección de caídas en el tráfico web mediante modelos estadísticos y redes neuronales

Trabajo de Fin de Grado

Convocatoria: Extraordinaria
Alumno/a: Antonio Cabrera Landín
Tutor/a: Stanislav Vakaruk
Grado: Ingeniería del Software

Agradecimientos

Quisiera agradecer a mi familia el apoyo recibido durante estos cinco años de carrera y a todos los profesores que me han acompañado, por todo lo que me han enseñado.

Índice general

Índice de figuras	3
Índice de tablas	5
Abstract	6
Glosario	8
Notación	10
1 Introducción	11
1.1 Justificación y contexto	11
1.2 Planteamiento del problema	11
1.3 Objetivos del trabajo	12
2 Estado de la cuestión	14
2.1 Evolución histórica	14
2.2 Marco teórico del trabajo	16
2.3 Trabajos relacionados	40
3 Aspectos metodológicos	42
3.1 Metodología	42
3.2 Tecnologías empleadas	45
4 Desarrollo del trabajo	48
4.1 Preparación y preprocesamiento de los datos	48
4.2 Análisis exploratorio de los datos	55
4.3 Marco común de evaluación	57
4.4 Modelos	60
4.5 Ajuste de hiperparámetros	70
4.6 Evaluación de los modelos	71
4.7 Aplicación de detección en tiempo real	74

5 Conclusiones	77
5.1 Conclusiones	77
5.2 Trabajo futuro	77
Referencias	79

Índice de figuras

2.1	Descomposición clásica de una serie temporal.	18
2.2	Ejemplo de serie temporal estacionaria y no estacionaria.	19
2.3	Gráficos ACF y PACF de una serie temporal.	20
2.4	<i>Outlier</i> aditivo.	24
2.5	<i>Outlier</i> de innovación.	25
2.6	<i>Outlier</i> de cambio de nivel.	25
2.7	<i>Outlier</i> de cambio temporal.	26
2.8	Ejemplo de arquitectura de red neuronal.	28
2.9	Función ReLU.	29
2.10	Función sigmoide.	30
2.11	Función tangente hiperbólica.	31
2.12	Esquema de una celda LSTM. Fuente: Zhang et al., 2023.	32
2.13	Arquitectura de una CNN.	36
2.14	Arquitectura de VGG-16. Fuente: github.com/spmallick/learnopencv	37
2.15	Arquitectura de DeepAnT. Fuente: Munir et al., 2019.	39
4.1	Serie temporal de los datos.	49
4.2	Ejemplos de caída total y caída parcial.	51
4.3	Visor de incidencias.	52
4.4	Distribución del tipo de caídas.	52
4.5	Comparación del volumen de peticiones de consulta entre un miércoles laborable y un miércoles festivo.	53
4.6	División cronológica del conjunto de datos en entrenamiento, validación y test. Las franjas rojas verticales indican las incidencias anotadas.	55
4.7	Patrón diario habitual del tráfico de consultas.	55
4.8	Mantenimiento programado de viernes a lunes.	56
4.9	Tráfico de un día representativo posterior al cambio estructural (sin incidencias registradas).	57
4.10	Descomposición estacional aditiva de la serie con período $m = 48$	61

4.11 Correlogramas ACF y PACF de la serie hasta 72 retardos.	62
4.12 Valores reales y predicciones de SARIMAX.	63
4.13 Arquitectura del modelo DeepAnT implementado.	64
4.14 Evolución de los errores de entrenamiento y validación del entrenamiento de DeepAnT.	66
4.15 Valores reales y predicciones de DeepAnT.	67
4.16 Arquitectura del modelo LSTM implementado.	68
4.17 Curvas de entrenamiento de una ejecución preliminar de la LSTM con 200 épocas fijas.	69
4.18 Curvas de entrenamiento de la LSTM con <i>early stopping</i> activado.	69
4.19 Valores reales y predicciones de la LSTM sobre una semana del conjunto de test.	70
4.20 Gráfico de Optuna en el que se muestra la importancia de cada hiperparámetro.	71
4.21 Detecciones de los cuatro modelos sobre un día con múltiples incidencias del conjunto de test.	72
4.22 Cuadro de mando de la aplicación de detección en tiempo real.	76

Índice de tablas

4.1	Estructura del conjunto de datos.	48
4.2	Estructura de las incidencias procesadas.	50
4.3	Estructura del conjunto de datos tras el preprocesamiento.	54
4.4	Métricas de predicción de SARIMAX sobre el conjunto de test.	63
4.5	Métricas de predicción de DeepAnT sobre el conjunto de test.	66
4.6	Métricas de predicción de LSTM sobre el conjunto de test.	70
4.7	Métricas de detección de los modelos sobre el conjunto de test.	71
4.8	Latencia mediana de detección y tiempo total de falsa alarma de los modelos sobre el conjunto de test.	73
4.9	Puntuación NAB de varios detectores de referencia sobre el <i>benchmark</i> oficial de Numenta (Ahmad et al., 2019).	74

Abstract

Resumen

Desarrollo de un sistema de alerta de caídas de aplicaciones basado en detección de anomalías en series temporales de tráfico. Se emplean datos reales de producción correspondientes al periodo 2023–2025 de una aplicación con servicio a nivel nacional, cuyo sistema de alarmas vigente, basado en reglas estadísticas rudimentarias, genera un volumen excesivo de falsos positivos. Tras una selección manual, se etiquetaron 355 incidencias entre caídas totales y parciales, las cuales constituyen la referencia objetiva sobre la que se evalúan los modelos.

Se implementan cuatro detectores en igualdad de condiciones: un modelo heurístico de referencia, un modelo SARIMAX, la red convolucional *DeepAnT* y una red recurrente LSTM. La comparativa sobre el conjunto de test sitúa a la LSTM como el detector más preciso ($F_1 = 0,675$ y puntuación NAB de 65,53), seguida muy de cerca por *DeepAnT*. El producto final integra los tres modelos de mejor desempeño (modelo base, *DeepAnT* y LSTM) en un sistema de monitorización en tiempo real con interfaz de cuadro de mando, aplicando además un criterio de consenso entre detectores que reduce los falsos positivos individuales.

Palabras clave: detección de anomalías, series temporales, SARIMAX, CNN, LSTM.

Abstract

Development of an application-outage alerting system based on anomaly detection over traffic time series. Real production data from 2023 to 2025 are used, belonging to a nationwide application whose existing rule-based alarm system generates an excessive number of false positives. After a manual selection, 355 incidents covering total and partial outages were labeled and used as the ground truth on which the models are evaluated.

Four detectors are implemented under identical conditions: a heuristic baseline, a SARIMAX model, the *DeepAnT* convolutional network and an LSTM recurrent network. The comparison over the test set places the LSTM as the most accurate detector ($F_1 = 0,675$ and NAB score of 65.53), closely

followed by DeepAnT. The final deliverable integrates the three best-performing models (baseline, DeepAnT and LSTM) into a real-time monitoring system with a dashboard interface, applying a consensus criterion among detectors that reduces individual false positives.

Keywords: anomaly detection, time series, SARIMAX, CNN, LSTM.

Glosario

ACF	Autocorrelation Function
PACF	Partial Autocorrelation Function
RB	Ruido Blanco
SR	Spectral Residual
AR	Autoregressive
MA	Moving Average
ARMA	Autoregressive Moving Average
ARIMA	Autoregressive Integrated Moving Average
SARIMA	Seasonal Autoregressive Integrated Moving Average
SARIMAX	Seasonal Autoregressive Integrated Moving Average with eXogenous factors
AO	Additive Outlier
IO	Innovational Outlier
LS	Level Shift
TC	Temporary Change
KNN	k-Nearest Neighbors
LOF	Local Outlier Factor
DNN	Deep Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
CNN	Convolutional Neural Network
SGD	Stochastic Gradient Descent
AIC	Akaike Information Criterion
ReLU	Rectified Linear Unit
ADF	Augmented Dickey-Fuller
KPSS	Kwiatkowski-Phillips-Schmidt-Shin

TP	True Positives (verdaderos positivos)
FN	False Negatives (falsos negativos)
FP	False Positives (falsos positivos)
MAE	Mean Absolute Error
RMSE	Root Mean Squared Error
MAPE	Mean Absolute Percentage Error
sMAPE	Symmetric Mean Absolute Percentage Error
NAB	Numenta Anomaly Benchmark
EDA	Exploratory Data Analysis
API	Application Programming Interface

Notación

$E(X)$	Esperanza de una variable aleatoria X
$\text{Var}(X)$	Varianza de una variable aleatoria X
$X \sim \text{RB}(\mu, \sigma^2)$	Variable aleatoria X que sigue una distribución de ruido blanco de media μ y varianza σ^2
X_t	Serie temporal indexada por el instante t
x_t	Observación de la serie en el instante t
$\hat{x}_t$	Predicción de la serie en el instante t
a_t	Término de ruido blanco en el instante t
γ_k	Autocovarianza de la serie a k retardos
ρ_k	Autocorrelación de la serie a k retardos
(p, d, q)	Órdenes autorregresivo, de integración y de media móvil del modelo ARIMA
$(P, D, Q)_m$	Órdenes estacionales del modelo SARIMA con período m
ϕ_i	Coefficientes autorregresivos del modelo ARIMA
θ_j	Coefficientes de media móvil del modelo ARIMA
ω_k	Coefficientes asociados a las variables exógenas
W	Tamaño de la ventana histórica de entrada de las redes neuronales
L	Horizonte multi-paso de predicción de la red LSTM
F_1	Puntuación F_1
R^2	Coefficiente de determinación

Capítulo 1

Introducción

En este capítulo se presenta el contexto general del trabajo. En primer lugar, se justifica el interés del problema de la detección de caídas en aplicaciones reales y se sitúa en el contexto actual de la industria del software. A continuación, se explica la problemática concreta que motiva el trabajo, asociada a un caso real de una empresa con sistema de alarmas rudimentario. Por último, se enumeran los objetivos específicos propuestos para abordar dicha problemática.

1.1 Justificación y contexto

Una de las prioridades históricas de la industria del software ha sido mantener la disponibilidad y rendimiento de sus servicios. Una caída del sistema puede provocar pérdidas críticas, no solo para el proveedor del servicio sino para todos sus clientes. Un ejemplo reciente es el incidente de CrowdStrike. Según el análisis de impacto de la firma Parametrix (Parametrix, 2024), se estima que el suceso ocasionó pérdidas de 5.400 millones de dólares para las empresas del Fortune 500.

En consecuencia, los ingenieros han implementado diversas técnicas para detectar, investigar la causa e incluso prevenir este tipo de caídas. Históricamente, se han empleado parámetros estadísticos estáticos como línea de referencia; sin embargo, el aumento exponencial de los volúmenes de datos y la adopción del Big Data en el sector ha generado una necesidad de métodos más sofisticados para la detección de estos valores atípicos.

1.2 Planteamiento del problema

La problemática se basa en un caso real de una empresa cuya aplicación da servicio a nivel nacional. Esta empresa desea saber si sus servidores están siendo atacados o si están fuera de servicio, para ello desarrolló un sistema de alarma basado en el número de peticiones en el tráfico. El problema viene del número de falsos positivos del sistema de monitoreo, la regla actual para decidir si un tráfico es anómalo es muy rudimentaria y por ende poco fiable.

Este trabajo se propone resolver este problema utilizando diferentes modelos de detección de valores extremos, yendo de los modelos clásicos de la estadística, a las técnicas más modernas basadas en Machine Learning.

El modelo deberá tener en cuenta la complejidad inherente de los datos de tráfico. En primer lugar, deberá considerar la estacionalidad anual y semanal, así como el tráfico reducido durante los días festivos. Además, tendrá que capturar el patrón diario de actividad, marcado por tráfico reducido durante la noche y picos pronunciados en horario laboral.

1.3 Objetivos del trabajo

El objetivo final del trabajo es desarrollar un sistema de detección y visualización de anomalías en el tráfico de peticiones de la aplicación en tiempo real. Este sistema deberá detectar las caídas del servicio empleando uno o varios modelos de detección de valores atípicos entrenados sobre la serie temporal de peticiones. Estos modelos reportarán los valores extremos teniendo en cuenta el momento del día y del año, así como el carácter laboral o festivo de cada jornada. La aplicación final contará con una interfaz visual que permita la monitorización del estado de los servidores.

Para lograr esta meta final, se proponen los siguientes objetivos:

- 1) Efectuar una limpieza y saneamiento del conjunto de datos en el que se aborden los valores repetidos, los posibles nulos y se estandaricen los tipos de cada columna.
- 2) Realizar un análisis exploratorio de los datos. Esto implica estudiar la salud y calidad de los datos, además de visualizar diversos gráficos para entender la naturaleza de la serie temporal.
- 3) Etiquetar manualmente las incidencias presentes en el conjunto de datos, con el fin de disponer de una referencia objetiva con la que evaluar los modelos.
- 4) Evaluar diferentes técnicas de detección de outliers en series temporales y concluir las ventajas y desventajas para el caso particular.
- 5) Implementar diferentes modelos de detección de outliers, tanto estadísticos como basados en Machine Learning.
- 6) Ajustar los hiperparámetros de los modelos implementados mediante un procedimiento automatizado de búsqueda.
- 7) Estudiar el desempeño de cada modelo mediante métricas de detección de anomalías como el F_1 y la puntuación NAB.

- 8) Integrar los modelos en una aplicación de monitorización en tiempo real que muestre el flujo de datos y alerte de las anomalías detectadas mediante un cuadro de mando.
- 9) Validar y verificar la solución final, asegurándose de que el programa cumple con los requisitos anteriores.

Capítulo 2

Estado de la cuestión

En este capítulo se sitúa el trabajo dentro de la literatura científica sobre detección de valores atípicos en series temporales. En primer lugar, se realiza un recorrido histórico por la evolución de las técnicas de detección, desde los criterios clásicos del siglo XIX hasta las arquitecturas modernas de *deep learning*. A continuación, en el marco teórico se introducen los conceptos matemáticos y los algoritmos sobre los que se sustenta el desarrollo del trabajo. Por último, se examinan publicaciones recientes que abordan problemas análogos en el ámbito de la detección de anomalías en tráfico de red.

2.1 Evolución histórica

El problema de los valores atípicos se remonta muy atrás en la historia de la estadística. Encuestas realizadas en los siglos XVIII y XIX ya mostraban que el problema era conocido por los matemáticos (Hawkins, 1980, pág. 9).

En el siglo XVIII, la solución más habitual de los investigadores consistía en eliminar las observaciones que parecían fuera de lugar. Este método rudimentario ya fue duramente criticado por el matemático Bernoulli (Hawkins, 1980, pág. 10).

En el siglo XIX, surgieron de la astronomía técnicas más elaboradas para la detección de *outliers*. En el año 1852, Benjamin Peirce propuso el primer criterio objetivo para el rechazo de valores atípicos, el cual cribaba las observaciones en función de si superaban cierto umbral probabilístico (Hawkins, 1980, pág. 10). Más tarde, en el año 1866, el matemático Chauvenet formuló un criterio más sencillo basándose en la distribución normal (Hawkins, 1980, pág. 10).

El siglo XX vio el nacimiento de los métodos de detección de *outliers* para datos temporales. En el año 1970, se postula la metodología Box-Jenkins (Box et al., 2008), la cual revoluciona la investigación de series temporales. Utilizando los modelos de Box-Jenkins, en el año 1972, A. J. Fox presenta dos modelos para la detección de *outliers* en series temporales (Fox, 1972). Un lustro después (1977), John W. Tukey publica su libro *Exploratory Data Analysis* (Tukey, 1977), en el que populariza el

diagrama de caja (*Box Plot*) y la regla del rango intercuartílico para la detección de *outliers*. Dos décadas después, se desarrollan algoritmos automatizados para detectar y minimizar el efecto de los *outliers* (Chang et al., 1988; Chen & Liu, 1993).

A finales de siglo (1998), los investigadores Edwin Knorr y Raymond Ng presentan su detección basada en distancia. En lugar de valerse de una distribución de probabilidad, evaluaron las distancias de los vecinos de cada observación (Knorr & Ng, 1998, pág. 393). La ventaja de este enfoque es que los algoritmos propuestos resultaban mucho más eficientes computacionalmente en comparación con los métodos anteriores (Knorr & Ng, 1998, pág. 392). En el siglo siguiente (2000), se perfecciona este enfoque con la publicación del algoritmo LOF (*Local Outlier Factor*) (Breunig et al., 2000). Este algoritmo solventa el problema de las agrupaciones de datos de diferentes densidades (Aggarwal, 2017, pág. 131).

En el año 2008 se produce un importante cambio de paradigma. En lugar de modelar qué es un dato anómalo, se propone aislar a los *outliers* utilizando el aprendizaje no supervisado (Liu et al., 2008, resumen). En concreto, se presenta el algoritmo de *Isolation Forest*, el cual reduce el cómputo a una complejidad lineal y además permite trabajar con datos de alta dimensionalidad (Liu et al., 2008, pág. 414).

En el año 2015, varias empresas liberan el código fuente de sus sistemas de detección. En primer lugar, Twitter desvela la técnica matemática detrás de su paquete de R (*AnomalyDetection*) (Kejariwal et al., 2015). Del mismo modo, Yahoo describe el funcionamiento de EGADS, un sistema automatizado de detección de *outliers* en series temporales a gran escala (Laptev et al., 2015). Así mismo, Netflix abre el código de su función de detección de *outliers* para *Big Data* (Wong et al., 2015).

En ese mismo año, los investigadores comienzan a utilizar redes neuronales recurrentes (RNN), específicamente la arquitectura LSTM, para la detección de anomalías en datos temporales (Malhotra et al., 2015). Para ello, entrenaron al modelo con datos libres de valores atípicos con el objetivo de utilizar los errores de predicción para modelar la aparición de *outliers*.

Por último, en el año 2018, se presenta DeepAnT. Esta metodología utiliza redes neuronales convolucionales profundas para encontrar anomalías (Munir et al., 2019). A diferencia de los modelos basados en LSTM, DeepAnT utiliza un aprendizaje no supervisado, permitiendo el uso de datos en bruto sin etiquetar. Además, requiere menos datos que sus predecesores, ya que se utilizó un 40 % de la muestra para entrenarlo (Munir et al., 2019, pág. 1992).

2.2 Marco teórico del trabajo

En este apartado se asentarán los conceptos teóricos necesarios para el posterior desarrollo de este trabajo. Además, se explicarán los algoritmos más relevantes de detección de *outliers*.

2.2.1 Valor extremo (*Outlier*)

Un *outlier* (también conocido como valor atípico, valor extremo o anomalía) es una observación que difiere significativamente de los datos. En la literatura es habitual utilizar la definición de Hawkins: «Una observación que se desvía tanto de otras observaciones como para despertar sospechas de que fue generada por un mecanismo diferente» (Hawkins, 1980, pág. 1).

Algunas de las causas más comunes de estos datos anómalos son: los errores de medición y registro, eventos inusuales como huelgas, catástrofes naturales o cambios repentinos en leyes y políticas.

Aunque históricamente se han percibido como datos erróneos que deterioraban los modelos de predicción, a menudo contienen información valiosa sobre el comportamiento del sistema. Por ejemplo, el estudio de valores atípicos ayuda a detectar enfermedades, detectar intrusiones en sistemas informáticos o a detectar actividad criminal (Aggarwal, 2017, págs.1-2).

2.2.2 Proceso estocástico

Un proceso estocástico es un fenómeno estadístico que evoluciona en el tiempo siguiendo unas leyes probabilísticas (Box et al., 2008, pág. 22). Matemáticamente, se define como una secuencia de variables aleatorias, ordenadas y equidistantes cronológicamente (Brockwell & Davis, 2016, pág. 6).

$$X_0, X_1, X_2, \dots \text{ donde } X_t \text{ es una variable aleatoria.}$$

Estos procesos estocásticos sirven para modelar comportamientos de observaciones reales y realizar predicciones. Además, permiten detectar valores anómalos.

Algunos ejemplos de estos procesos son las cadenas de Markov, las cuales se pueden utilizar para sugerir texto en un teclado virtual o la caminata aleatoria (*Random Walk*), que se utiliza para modelar la bolsa de forma rudimentaria.

2.2.3 Series temporales

Una serie temporal es un conjunto de observaciones x_t , cada una registrada en un tiempo específico t (Brockwell & Davis, 2016, pág. 1). Ejemplos de series temporales pueden ser los indicadores macroeconómicos de un país o las temperaturas medias anuales del planeta.

También se puede definir como un período muestral de un proceso estocástico (Box et al., 2008, pág. 23).

Habitualmente, los datos observados no son independientes entre sí. Analizar esta dependencia nos permite entender el comportamiento de los datos o incluso realizar predicciones. (Box et al., 2008, pág. 1)

Tipos de series temporales

Existen diferentes tipos de series temporales:

Una serie temporal puede ser continua o discreta: las series discretas recogen sus datos en momentos específicos, mientras que las continuas registran las observaciones de forma continua en un intervalo de tiempo (Brockwell & Davis, 2016, pág. 1). En este trabajo se estudiarán series temporales discretas cuyos datos han sido recogidos en intervalos fijos de tiempo.

Una serie temporal puede ser univariante o multivariante: Cuando las observaciones corresponden a una única variable, decimos que la serie es univariante. Cuando corresponden a más de una, entonces decimos que son multivariantes.

2.2.4 Modelo de descomposición clásico

Para poder analizar las series temporales, el enfoque clásico (Brockwell & Davis, 2016, sec. 1.5) consiste en descomponer la serie en diferentes componentes.

Tendencia

La tendencia (m_t), marca la evolución a lo largo del tiempo. Por ejemplo, podría explicar el crecimiento del valor de las acciones de una empresa.

Estacionalidad

La estacionalidad (s_t) se define como un patrón que se repite con cierta periodicidad, lo que lo hace predecible. Por ejemplo, podría explicar el incremento de turistas durante la temporada alta.

Ruido

El ruido (Y_t) engloba todos los factores desconocidos que no se pueden explicar. Por ejemplo, podría recoger una huelga o una catástrofe.

El modelo aditivo del enfoque clásico se explicaría mediante la siguiente ecuación:

$$X_t = m_t + s_t + Y_t \quad (2.1)$$

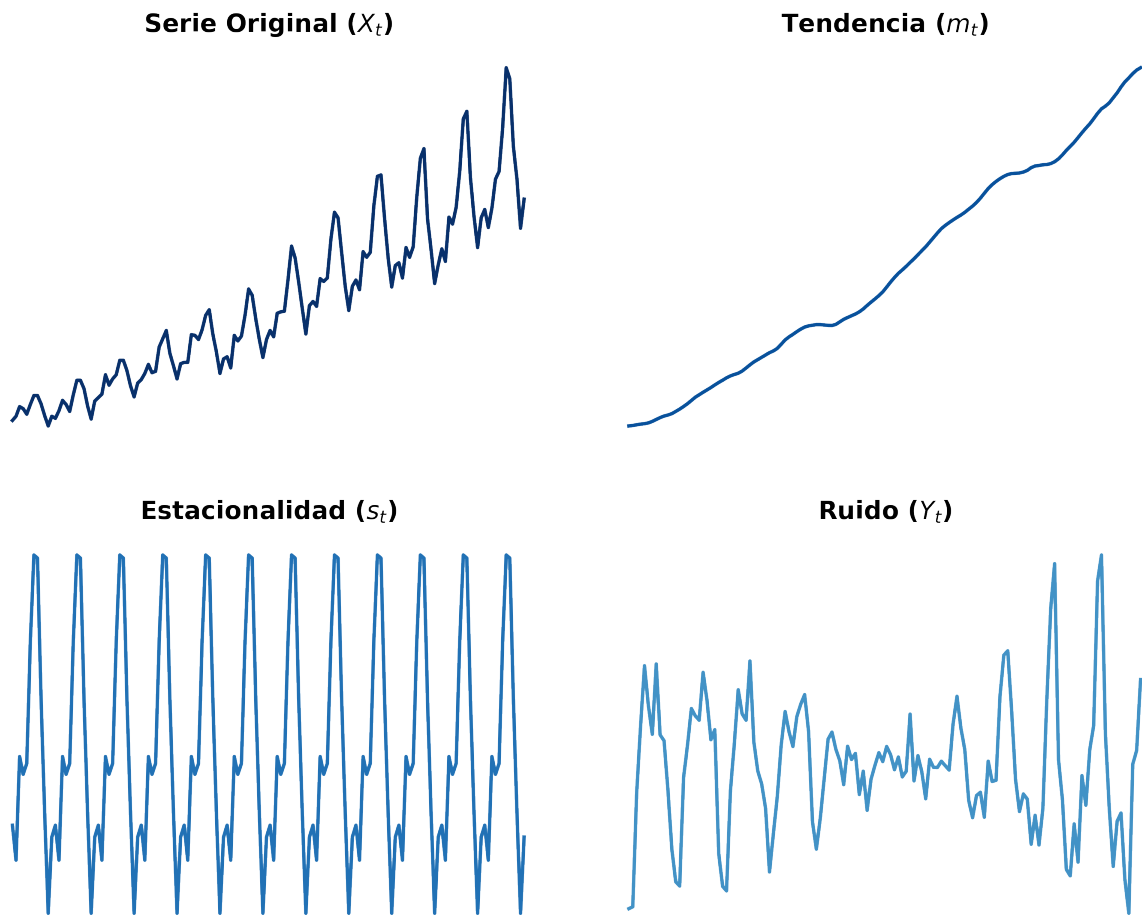


Figura 2.1: Descomposición clásica de una serie temporal.

2.2.5 Estacionariedad

Decimos que un proceso estocástico es estacionario cuando sus propiedades estadísticas (media, varianza y covarianza) se mantienen similares en el tiempo (Brockwell & Davis, 2016, pág. 13). Por lo tanto, un proceso estacionario no presenta tendencia ni estacionalidad debido a que ambas componentes varían las propiedades estadísticas ya comentadas. Cabe resaltar que si un proceso estocástico es estacionario también lo serán las series temporales derivadas de este.

Matemáticamente, un proceso estocástico (débil)¹ cumple las siguientes propiedades (Brockwell & Davis, 2016, def. 1.4.2; Box et al., 2008, sec. 2.1.2):

- $E(X_t) = \mu$ es independiente del tiempo. Es decir, $E(X_t) = \mu$ para todo t .
- $\text{Cov}(X_t, X_{t+k})$ es independiente del tiempo. Es decir, $\text{Cov}(X_t, X_{t+k}) = \gamma_k$ para $k = \pm_1, \pm_2, \dots$

De estas dos propiedades, se deduce que la varianza también se mantiene constante.

¹Existen dos tipos de estacionariedad: la fuerte y la débil. En este trabajo, cuando se haga referencia al concepto estacionariedad, se estará refiriendo a la débil.

- $\text{Var}(X_t) = \sigma^2$ para todo t .

En la figura 2.2 la gráfica de la izquierda es estacionaria, ya que su varianza y media se mantienen constantes. Sin embargo, la otra gráfica no tiene media fija ya que presenta tendencia positiva.

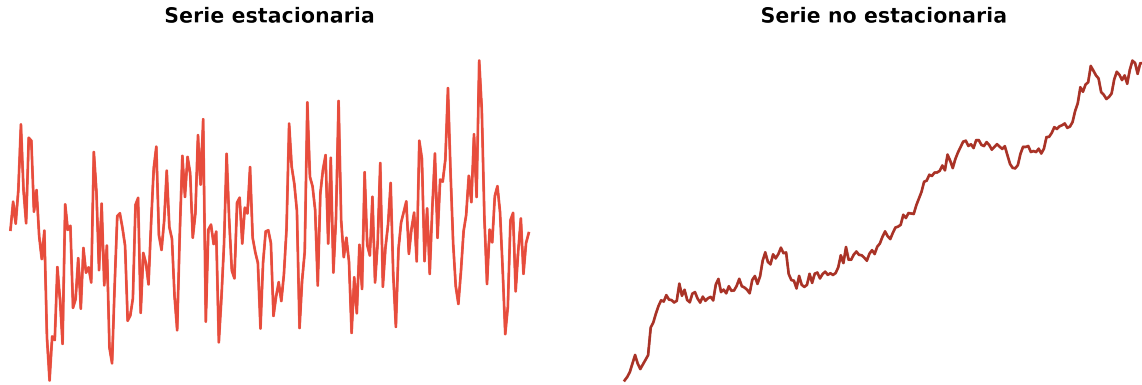


Figura 2.2: Ejemplo de serie temporal estacionaria y no estacionaria.

2.2.6 Autocorrelación

Retardo

El retardo o *lag* es un intervalo de tiempo k el cual separa una observación x_t de otra x_{t+k} .

Coefficiente de autocovarianza

Sea X_t un proceso estacionario, su coeficiente de autocovarianza γ_k en el retardo k recoge todas las autocovarianzas del proceso (Box et al., 2008, pág. 25).

$$\gamma_k = \text{Cov}(X_t, X_{t+k}) = E[X_t - \mu][X_{t+k} - \mu] \quad (2.2)$$

Coefficiente de autocorrelación

El coeficiente de autocorrelación ρ_k mide el grado de interdependencia lineal existente entre dos variables aleatorias del proceso estocástico separadas por un retardo k (Box et al., 2008, pág. 25).

$$\rho_k = \frac{\gamma_k}{\gamma_0} \quad (2.3)$$

Función de autocorrelación (ACF)

La función de autocorrelación (ACF) de un proceso estacionario es una función de k que recoge todos los coeficientes de autocorrelación ρ_k del proceso (Box et al., 2008, pág. 29). La gráfica de esta función de autocorrelación se llama correlograma.

El correlograma puede mostrar de forma evidente si una serie temporal es estacionaria.

Función de autocorrelación parcial (PACF)

Mientras que el gráfico de la función de autocorrelación simple (ACF) muestra la correlación entre dos observaciones y todas las intermedias, la función de autocorrelación parcial descarta la dependencia lineal de estos valores intermedios (Peña, 2010, pág. 128).

Al igual que con el correlograma de la función ACF, el gráfico PACF también sirve para determinar si una serie es estacionaria.

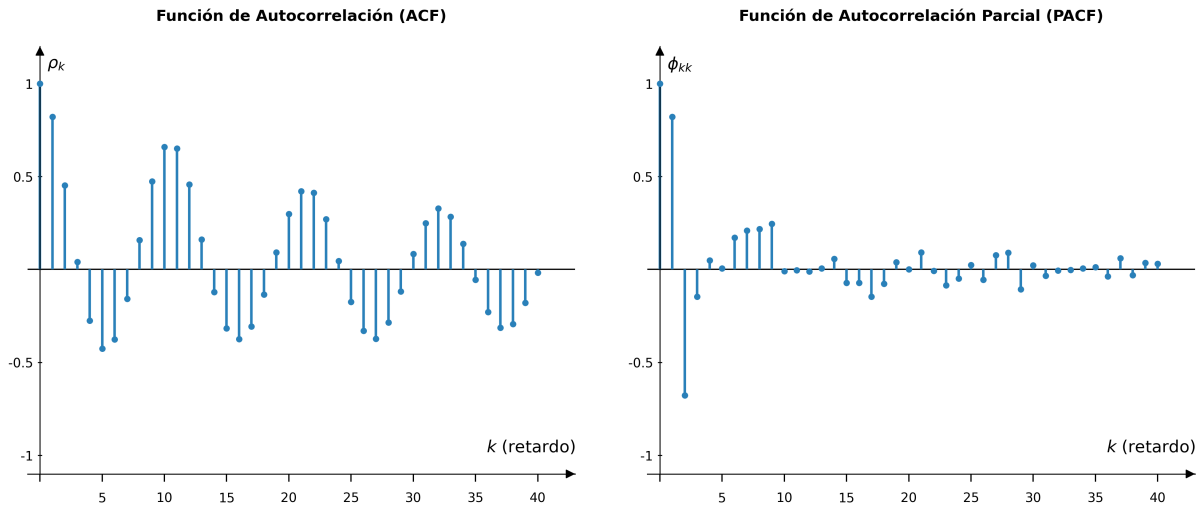


Figura 2.3: Gráficos ACF y PACF de una serie temporal.

2.2.7 Familia de modelos ARIMA

A continuación se exponen algunos de los modelos del libro (Box et al., 2008) para la predicción de series temporales.

Modelo autorregresivo (AR)

Un modelo autorregresivo de orden p , denotado como $AR(p)$, es un modelo estocástico en que el valor actual viene determinada por los valores pasados y por el ruido.

Asumiendo sin pérdida de generalidad que $\mu = 0$, el modelo autorregresivo se explica mediante la siguiente ecuación (Shumway & Stoffer, 2025, def. 3.1):

$$x_t = \phi_1 x_{t-1} + \phi_2 x_{t-2} + \cdots + \phi_p x_{t-p} + a_t \quad (2.4)$$

Si la media fuese $E(x_t) = \mu \neq 0$, entonces reemplazaríamos x_t por $x_t - \mu$:

$$x_t - \mu = \phi_1 (x_{t-1} - \mu) + \phi_2 (x_{t-2} - \mu) + \cdots + \phi_p (x_{t-p} - \mu) + a_t \quad (2.5)$$

- Los parámetros $(\phi_1, \phi_2, \dots, \phi_p)$ conforman el vector de parámetros autorregresivos y son los que asignan relevancia a cada evento pasado.
- El término a_t hace referencia al error aleatorio del modelo. En concreto, es una variable aleatoria que sigue una distribución de ruido blanco de media nula y varianza constante $a_t \sim \text{RB}(0, \sigma^2)$.

Para hallar un orden p óptimo, se puede recurrir al gráfico PACF (Shumway & Stoffer, 2025, pág. 108).

Modelo de medias móvil (MA)

El modelo de media móvil explica cada observación x_t utilizando el ruido blanco a_t de las observaciones actual y pasadas (Shumway & Stoffer, 2025, pág. 92). Concretamente, un modelo de media móvil de orden q se define mediante la siguiente forma (Shumway & Stoffer, 2025, def. 3.3):

$$x_t = a_t + \theta_1 a_{t-1} + \theta_2 a_{t-2} + \dots + \theta_q a_{t-q} \quad (2.6)$$

Donde las variables aleatorias a_t siguen una distribución de ruido blanco $a_t \sim \text{RB}(0, \sigma^2)$.

Para el modelo de media móvil, nos podemos apoyar en el gráfico ACF para determinar un orden q apropiado (Shumway & Stoffer, 2025, pág. 108).

ARMA

La combinación de los dos modelos anteriores (2.4, 2.6) se llama modelo ARMA de orden p y q .

$$x_t = \mu + \phi_1(x_{t-1} - \mu) + \dots + \phi_p(x_{t-p} - \mu) + a_t + \theta_1 a_{t-1} + \dots + \theta_q a_{t-q} \quad (2.7)$$

El requisito para poder construir este tipo de modelo es que la serie sea estacionaria (Shumway & Stoffer, 2025, def. 3.5).

ARIMA

El modelo autorregresivo integrado de media móvil, denotado como $\text{ARIMA}(p, d, q)$, consiste en una ampliación del modelo $\text{ARMA}(p, q)$ y está diseñado para sortear este requisito de estacionariedad de las series. Para ello, se diferencia la serie temporal d veces hasta que sea estacionaria. El proceso de diferenciación consistirá en restar a las observaciones las anteriores (Shumway & Stoffer, 2025, pág. 145):

$$\nabla x_t = x_t - x_{t-1} \quad (2.8)$$

La I de ARIMA viene de Integrado, debido a que para devolver los datos a su forma original deberemos integrar d veces de vuelta.

La fórmula del modelo ARIMA es similar a la del modelo ARMA(p, q) (2.7)

$$x_t = \mu + \phi_1(x_{t-1} - \mu) + \cdots + \phi_p(x_{t-p} - \mu) + a_t + \theta_1 a_{t-1} + \cdots + \theta_q a_{t-q} \quad (2.9)$$

La diferencia entre estos dos modelos es el parámetro d ya mencionado.

SARIMA

El modelo autorregresivo integrado de media móvil estacional (SARIMA), es una extensión del ARIMA utilizado cuando las series temporales presentan estacionalidad, es decir, cuando se repiten patrones a lo largo de meses o trimestres.

Se denota como SARIMA(p, d, q) $\times$ (P, D, Q) $_m$ (Brockwell & Davis, 2016, pág. 117; Hyndman & Athanasopoulos, 2018, sec. 8.9). Los parámetros p, d, q son los mismos que en ARIMA; sin embargo, se añaden las componentes (P, D, Q) que hacen referencia a la parte estacional de la serie. El símbolo m es la frecuencia estacional, es decir, el número de periodos en cada ciclo. Por ejemplo, si los datos tienen una periodicidad diaria y $m = 7$, entonces el patrón estacional es semanal. Por otro lado, si la periodicidad es mensual y $m = 12$, el patrón es anual.

SARIMAX

El modelo SARIMAX representa la evolución final de esta jerarquía al introducir variables externas o exógenas (denotadas por la letra X). Este enfoque es fundamental cuando la serie temporal no solo depende de su propio pasado, sino también de factores externos independientes como, por ejemplo, los días festivos.

Matemáticamente, el modelo SARIMAX combina un componente de regresión lineal para las variables exógenas con el modelado de los errores mediante una estructura SARIMA (Hyndman & Athanasopoulos, 2018, cap. 9). Esto permite que el algoritmo no solo capture la inercia y la estacionalidad, sino que también cuantifique el impacto de eventos específicos.

2.2.8 Metodología de Box-Jenkins

La metodología de Box-Jenkins constituye un marco iterativo para la construcción de modelos de series temporales. A diferencia de otros enfoques, no asume un modelo a priori, sino que utiliza los datos para determinar la estructura más adecuada mediante un proceso cíclico de tres etapas (Box et al., 2008, pág. 193):

- 1) **Identificación:** En esta fase se verifica el requisito de estacionariedad. Si la serie no es estacionaria, se aplican transformaciones o diferenciaciones (el componente d de ARIMA).

Posteriormente, se utilizan las funciones de autocorrelación (ACF) y autocorrelación parcial (PACF) para proponer los órdenes potenciales de los componentes AR y MA.

- 2) **Estimación:** Una vez seleccionados los órdenes (p, d, q) , se calculan los coeficientes de los parámetros mediante métodos estadísticos como la máxima verosimilitud o mínimos cuadrados no lineales.
- 3) **Evaluación diagnóstica:** Se comprueba si el modelo es adecuado analizando los residuos. Para que el modelo sea válido, los residuos deben comportarse como ruido blanco, es decir, no deben presentar patrones ni autocorrelaciones significativas. Si el diagnóstico falla, se regresa a la etapa de identificación para ajustar los parámetros.

2.2.9 Tipos de *Outliers*

Los *outliers* son perniciosos para el análisis estadístico, especialmente para la metodología de Box-Jenkins. Esto se debe a que incrementan la varianza artificialmente. En consecuencia, los *outliers* pueden distorsionar los gráficos ACF y PACF, propiciando malas decisiones en los investigadores. Además, introducen sesgos a las técnicas utilizadas para estimar los parámetros ϕ, θ de los modelos ARIMA, lo cual perjudica notablemente su precisión.

A continuación se describe la taxonomía de los valores anómalos utilizada en el marco de trabajo de los modelos ARIMA.

***Outlier* aditivo (AO)**

El *outlier* aditivo (*Additive Outlier*) se considera un error aislado el cual produce un pico en la serie (Chen & Liu, 1993, pág. 285). Solo se observa en un momento concreto (Box et al., 2008, pág. 537), por lo que no afecta a las observaciones futuras.

Un ejemplo de AO sería un error de medición en los datos de la serie temporal.

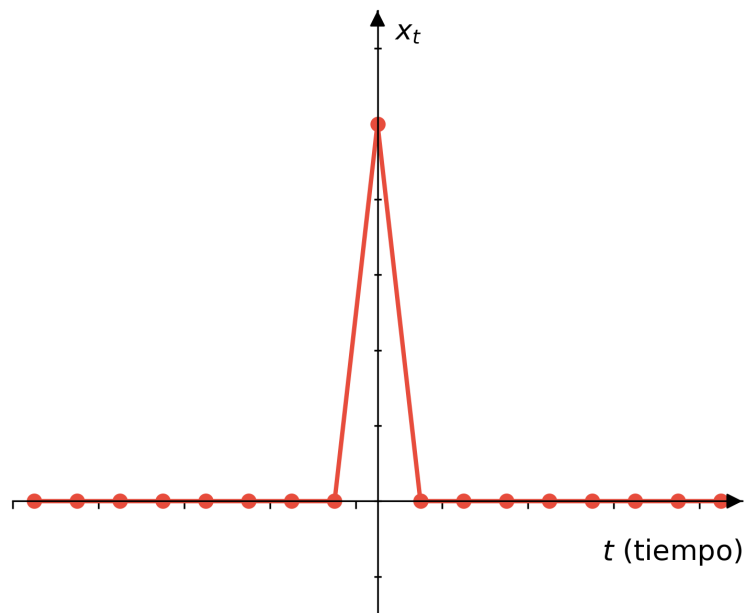


Figura 2.4: *Outlier* aditivo.

***Outlier* de innovación (IO)**

El *outlier* de innovación (*Innovation Outlier*) no solo afecta a la serie en un momento determinado, sino que además influye en futuras observaciones (Box et al., 2008, pág. 537). Los cambios introducidos a la serie no son un escalón perfecto, sino que dependen de la complejidad y la naturaleza del proceso.

Un ejemplo de este tipo de *outlier* podría ser las declaraciones de un actor. Este suceso puede alterar las mediciones en la taquilla de la película en la que actúa.

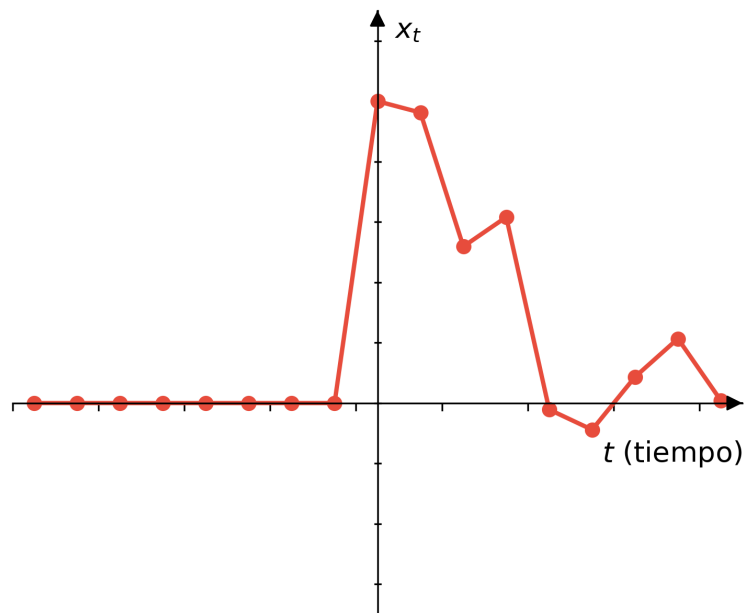


Figura 2.5: *Outlier* de innovación.

Cambio de nivel (LS)

El cambio de nivel (*Level Shift*), es un evento que altera la serie de forma abrupta. Produce un cambio de nivel escalonado el cual es permanente (Chen & Liu, 1993, pág. 285).

Un ejemplo de cambio de nivel podría ser, en la medición de la velocidad de un coche de carreras, la instalación de un motor más potente.

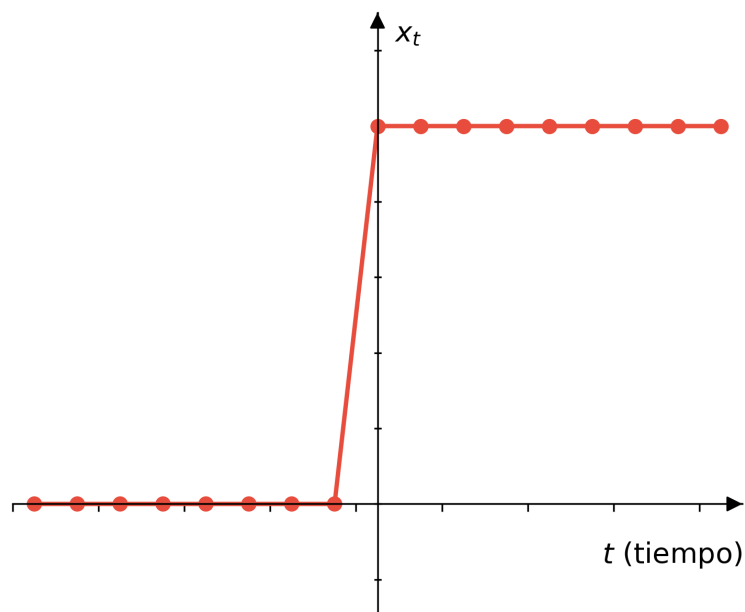


Figura 2.6: *Outlier* de cambio de nivel.

Cambio temporal (TC)

El cambio temporal (*Temporary Change*) es similar al cambio de nivel (LS). La diferencia radica en que los efectos del TC en la serie son temporales mientras los del LS son permanentes.

Este tipo de valor extremo se observa como un cambio brusco en los datos que modifica la serie, pero que va decayendo de forma gradual hasta que la serie recobra sus niveles originales (Chen & Liu, 1993, pág. 285).

Un ejemplo de TC podría ser una guerra que afecta al precio del barril de petróleo, pero que al finalizar devuelve su coste a su estado anterior.

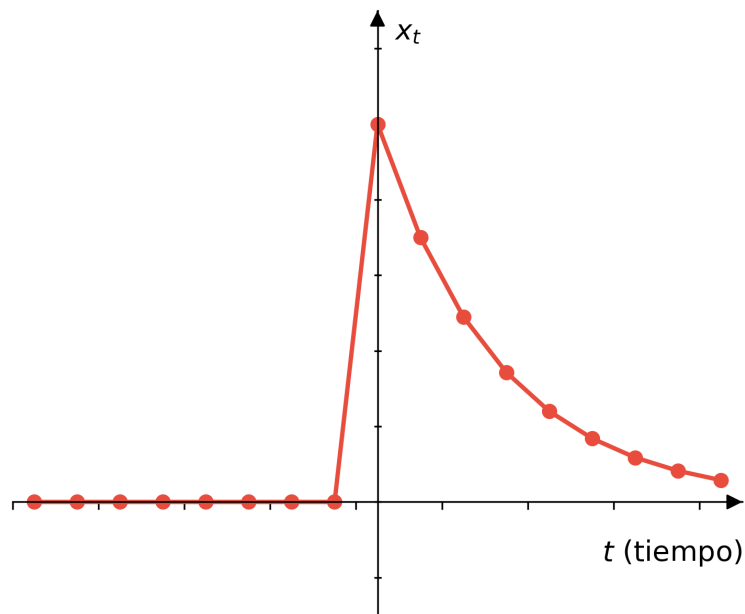


Figura 2.7: *Outlier* de cambio temporal.

2.2.10 Procedimiento iterativo para la detección de *outliers*

El procedimiento iterativo para detección de *outliers* tiene por objetivo detectar valores extremos de tipo IO y AO, además de estimar de forma efectiva los parámetros del modelo ARIMA en presencia de *outliers* (Chang et al., 1988, pág. 193).

Consta de dos etapas las cuales se repiten de forma iterativa. Una de detección de *outliers* y otra de estimación de parámetros.

Etapas 1. Detección de *outliers*

Primero, se ajusta el modelo a la serie temporal bajo el supuesto de que no existen valores extremos (Chang et al., 1988, pág. 197).

Una vez se tiene este modelo base, se procede a la detección de los *outliers* (Chang et al., 1988, sec. 3.1). Para ello, en cada instante t de la serie, se computan dos estadísticos: uno diseñado para detectar valores atípicos de tipo IO, denotado como $\hat{\lambda}_{1,t}$, y otro para detectar valores atípicos AO, denotado como $\hat{\lambda}_{2,t}$.

A continuación, de todos los estadísticos calculados se selecciona el mayor. Si el valor máximo supera un nivel crítico predeterminado (habitualmente 3.0, 3.5 o 4.0), diremos que existe un *outlier* en ese instante t y del tipo correspondiente. Una vez que ha sido detectado, se estima su impacto o magnitud (ω).

Finalmente, se elimina el efecto de dicho *outlier* de los residuos del modelo y se recalcula su varianza. Este procedimiento se repite de forma iterativa con los nuevos errores ajustados, finalizando el proceso únicamente cuando no se detecten más valores extremos.

Etapas 2. Estimación de parámetros

En esta etapa del procedimiento, se recalculan los parámetros autorregresivos ($\phi_1, \dots, \phi_p$) y de medias móviles ($\theta_1, \dots, \theta_q$) del modelo (Chang et al., 1988, sec. 3.2). Si en la etapa anterior se detectaron k valores atípicos, se procede a estimar sus magnitudes ($\omega_1, \dots, \omega_k$) junto con los parámetros del modelo de forma simultánea.

Con estos nuevos parámetros, se repite la etapa 1 de detección de *outliers* de nuevo. Si no se encuentran nuevos valores extremos, se para con el procedimiento de forma definitiva. En el caso contrario, se siguen con la etapa 2 de nuevo.

Este algoritmo fue el primer método automatizado para la detección de *outliers* en modelos ARIMA. Sin embargo, contaba con el inconveniente de detectar los *outliers* de forma individual. Esto se debía a que si se encontraban varios valores extremos consecutivos, detectarlos uno por uno causaba un «efecto de enmascaramiento» (Chen & Liu, 1993, pág. 284).

2.2.11 Procedimiento de estimación conjunta

Cinco años después de la publicación del primer método (Chang et al., 1988), se desarrolló el procedimiento de estimación conjunta (Chen & Liu, 1993).

El método surge como evolución del anterior, con el propósito de solventar el problema de enmascaramiento producido por la detección individual de valores extremos. Para ello, se estiman los efectos de los *outliers* ($\omega_1, \dots, \omega_k$) de forma conjunta utilizando un modelo de regresión (Chang et al., 1988, pág. 287). Además, se proponen dos nuevos tipos de valores atípicos: los de tipo LS y TC (Chen & Liu, 1993, pág. 285).

2.2.12 *Machine Learning*

Se conoce como *machine learning* a las técnicas por las cuales los sistemas informáticos aprenden a partir de los datos (Goodfellow et al., 2016, pág. 8). En lugar de que un humano programe explícitamente una tarea, es el propio ordenador el que averigua la solución.

Los algoritmos de *machine learning* se emplean en multitud de escenarios, como en los algoritmos de recomendación, en la toma de decisiones de una empresa o en la visión por computador.

2.2.13 Red Neuronal

Una red neuronal es un modelo computacional de *machine learning* el cual se inspira en el funcionamiento del propio cerebro humano (Aggarwal, 2017, pág. 98). La red está formada por neuronas las cuales se organizan en capas. Cada neurona recibe la información de las neuronas de la capa anterior mediante conexiones que tienen un peso asociado. La neurona computa la suma ponderada de estas entradas y le aplica una función de activación no lineal. A la primera capa de la red se la conoce como capa de entrada, a las siguientes capas ocultas y a la final, capa de salida.

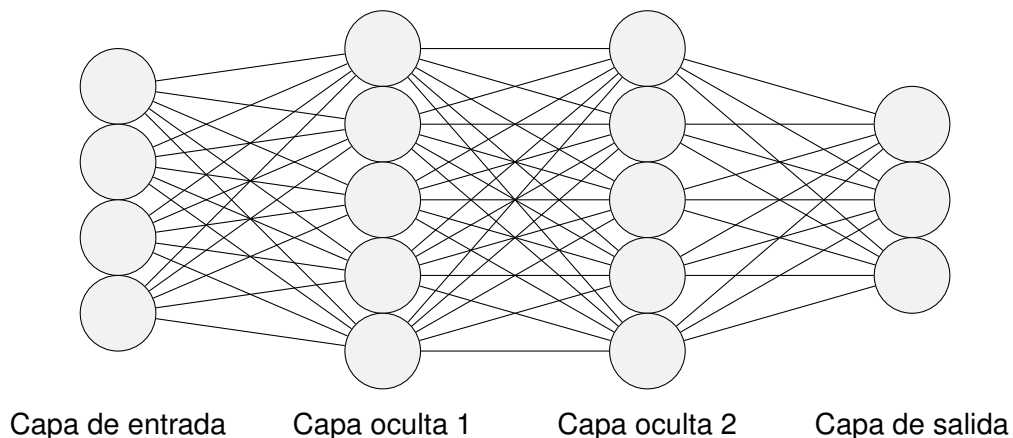


Figura 2.8: Ejemplo de arquitectura de red neuronal.

Para que la red sea efectiva, se deben encontrar unos pesos adecuados. Para ello se entrena a la red neuronal, primero calculando el error de predicción con una función de pérdida y después corrigiéndolo mediante algoritmos matemáticos como el descenso gradiente y la retropropagación (*backpropagation*).

2.2.14 Función de activación

Una función de activación es una operación matemática, generalmente no lineal, que se le aplica a las salidas de las neuronas de las redes neuronales (Goodfellow et al., 2016, pág. 172). El propósito de estas funciones es introducir no linealidad al modelo: si la red permanece completamente lineal, el uso

de más capas será redundante y además no podrá aprender comportamientos complejos (Aggarwal, 2023, sec. 1.5).

Existen varios tipos de funciones de activación, a continuación se explicarán algunas de las más habituales.

ReLU

Se trata de la función de activación más recomendada y utilizada por defecto para las capas ocultas de la red. La Unidad Lineal Rectificada (ReLU, por sus siglas en inglés) es una función definida por partes que devuelve cero si la entrada es negativa, y devuelve el mismo valor de la entrada si esta es positiva:

$$f(x) = \max(0, x) \quad (2.10)$$

Su popularidad se debe a que es extremadamente eficiente a nivel computacional y mitiga en gran medida el problema del desvanecimiento del gradiente (*vanishing gradient*), lo que permite entrenar redes neuronales profundas con mayor rapidez.

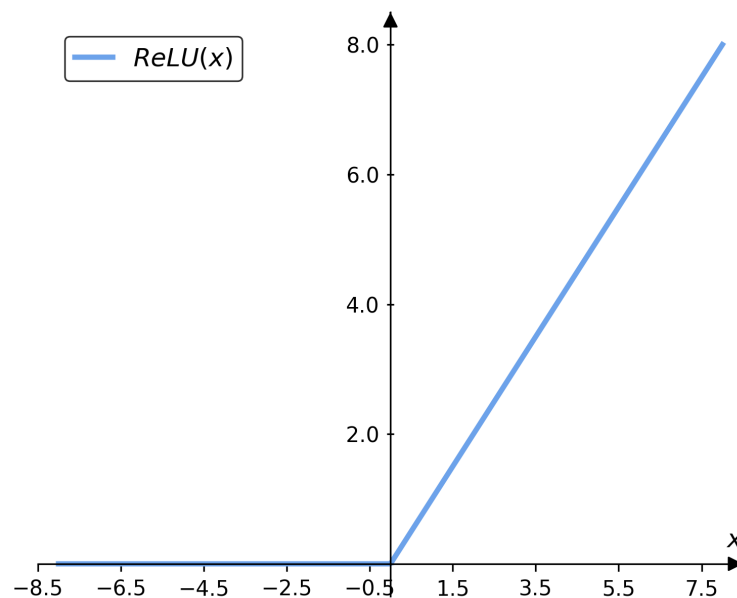


Figura 2.9: Función ReLU.

Sigmoide

La función sigmoide transforma cualquier valor real en un valor comprendido en el intervalo $[0, 1]$. Su expresión matemática es la siguiente:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.11)$$

El mayor uso que se le da a esta función es en la capa de salida. Al devolver valores entre 0 y 1, se puede interpretar como la probabilidad y confianza de predicción.

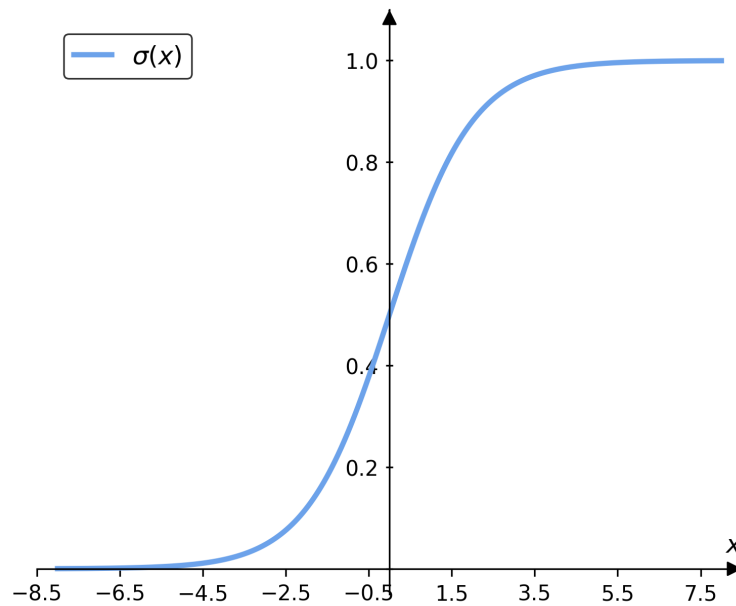


Figura 2.10: Función sigmoide.

Tangente hiperbólica

La función tangente hiperbólica es una versión reescalada y desplazada de la función sigmoide. A diferencia de esta última, la tangente hiperbólica puede devolver valores negativos. Esto se debe a que su intervalo ahora queda comprendido entre $[-1, 1]$.

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2.12)$$

El hecho de estar centrada en cero facilita la convergencia del algoritmo de optimización durante el entrenamiento. Aunque presenta un mejor desempeño que la función sigmoide en las capas ocultas, sigue sufriendo el problema del desvanecimiento del gradiente en los extremos de la función.

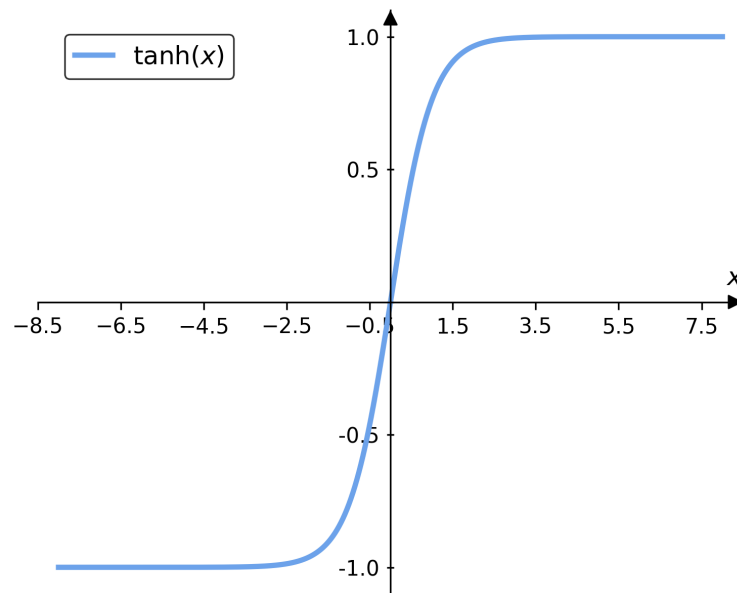


Figura 2.11: Función tangente hiperbólica.

2.2.15 Redes neuronales recurrentes (RNN)

Las redes neuronales recurrentes son un tipo de red neuronal especializada en el procesamiento de datos secuenciales (Goodfellow et al., 2016, pág. 373). Por ello, se emplean para el procesamiento del lenguaje natural, reconocimiento de voz o para predecir series temporales.

Este tipo de red utiliza conexiones de retroalimentación para mantener una «memoria» a lo largo del tiempo. Sin embargo, las redes neuronales recurrentes sufren del problema de desvanecimiento y explosión de gradiente. Cuando el gradiente se prolonga en el tiempo, tienden a desvanecerse o explotar (Goodfellow et al., 2016, pág. 401). Esto complica el aprendizaje de interacciones lejanas en el tiempo.

2.2.16 LSTM

Las LSTM (Hochreiter & Schmidhuber, 1997) son un tipo de RNN que aborda el problema del desvanecimiento del gradiente ² mediante la introducción de celdas de memoria. Las celdas separan el flujo de información en un estado interno (C_t) y un estado oculto (H_t). Esta arquitectura tiene dos objetivos: por un lado, proteger la memoria a largo plazo permitiendo que el estado interno se actualice mediante operaciones lineales aditivas (mitigando así el desvanecimiento); y por otro, utilizar el estado oculto para filtrar dicha memoria y exponer solo la información relevante en cada paso temporal.

²En contraste, para atajar el problema de la explosión del gradiente, se emplea habitualmente la técnica de recorte de gradiente durante el entrenamiento (Goodfellow et al., 2016, sec. 10.11.2).

Funcionamiento de las celdas de memoria

Las celdas de memoria están construidas mediante un sistema de compuertas multiplicativas que controlan qué información entra, sale o se olvida. Además, regulan cómo se utiliza la memoria para la salida.

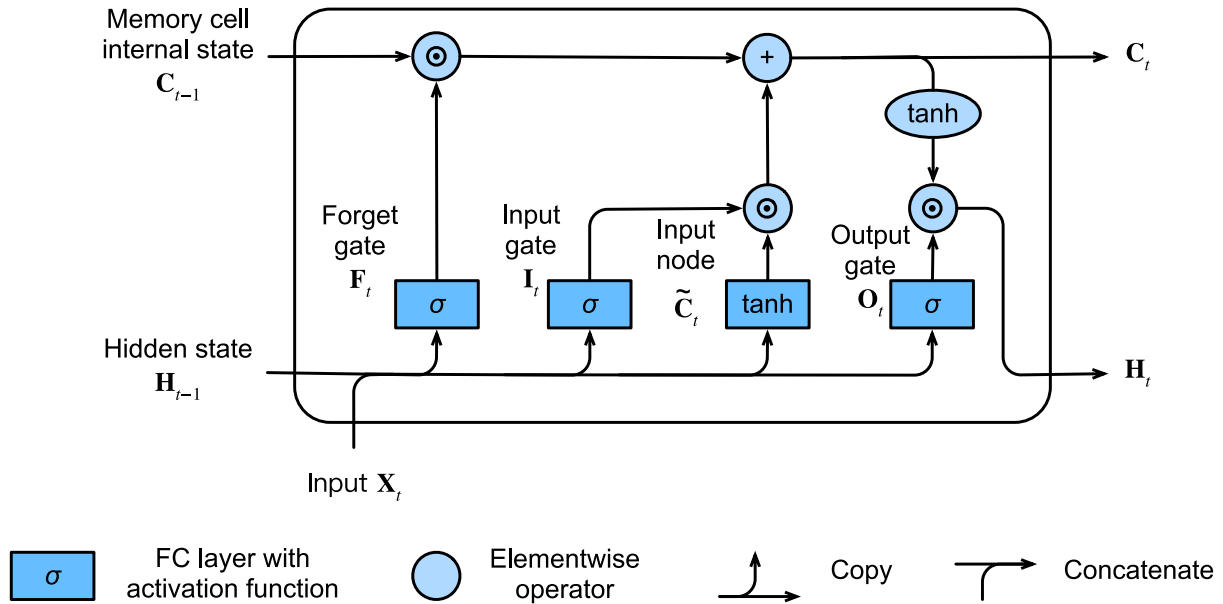


Figura 2.12: Esquema de una celda LSTM. Fuente: Zhang et al., 2023.

La figura 2.12 ilustra el funcionamiento interno de una celda de memoria. La línea superior muestra el flujo del estado interno de la celda (C_t), que se interpreta como la memoria a largo plazo. Por otro lado, la línea inferior sigue al estado oculto de la celda (H_t), el cual constituye la memoria a corto plazo. Además de estas dos flechas, vemos operaciones elemento a elemento, funciones de activación y las puertas anteriormente comentadas.

El funcionamiento de la celda de memoria se explica en tres pasos:

- 1) Se decide que información de la memoria a largo plazo retener mediante la puerta de olvido F_t .
- 2) Se decide que información añadir a la memoria a largo plazo mediante la puerta de entrada I_t .
- 3) Se decide que información de la memoria a largo plazo es relevante en el corto plazo H_t a través de la puerta de salida O_t .

En primer lugar, en la línea inferior, se recibe la memoria a corto plazo de la celda anterior H_{t-1} , a la que se le concatena una información nueva X_t . La primera compuerta que atraviesa este vector concatenado $[H_{t-1}, X_t]$ es la compuerta de olvido F_t . Al hacerlo, el vector se multiplica por una

matriz de pesos W_f y se le suma un vector de sesgo b_f . Por último, el cálculo anterior se pasa por una función de activación sigmoide que producirá un vector cuyos valores están entre 0 y 1.

$$\mathbf{F}_t = \sigma(W_f \cdot [\mathbf{H}_{t-1}, \mathbf{X}_t] + b_f) \quad (2.13)$$

Este vector se multiplica elemento a elemento con la memoria a largo plazo de la celda anterior $\mathbf{C}_{t-1}$, lo que reduce su impacto produciendo el efecto de olvido. Esto se debe a que los componentes del vector son valores entre 0 y 1 que actúan como el porcentaje de información que se retiene de la memoria a largo plazo.

$$\mathbf{F}_t \odot \mathbf{C}_{t-1} \quad (2.14)$$

La segunda compuerta corresponde a la puerta de entrada $\mathbf{I}_t$. Esta determina qué parte de una memoria candidata $\tilde{\mathbf{C}}_t$ se va a añadir al estado interno de la celda. Matemáticamente, se modela de la misma forma que la compuerta anterior (2.13).

$$\mathbf{I}_t = \sigma(W_i \cdot [\mathbf{H}_{t-1}, \mathbf{X}_t] + b_i) \quad (2.15)$$

A la memoria candidata $\tilde{\mathbf{C}}_t$ se le conoce como nodo de entrada y se calcula teniendo en cuenta la información de entrada $\mathbf{X}_t$ y el estado oculto anterior $\mathbf{H}_{t-1}$. Las operaciones matemáticas son similares a las de las otras puertas (2.13, 2.15); sin embargo, se cambia la función de activación a la tangente hiperbólica. El motivo de utilizar la tangente hiperbólica es que esta función devuelve valores entre -1 y 1, lo que permite al estado interno $\mathbf{C}_t$ tomar valores negativos e invertir información si es necesario.

$$\tilde{\mathbf{C}}_t = \tanh(W_c \cdot [\mathbf{H}_{t-1}, \mathbf{X}_t] + b_c) \quad (2.16)$$

En definitiva, el cálculo de la memoria a largo plazo $\mathbf{C}_t$ es el siguiente: se computa el vector de olvido $\mathbf{F}_t$ y se multiplica elemento a elemento con el estado anterior $\mathbf{C}_{t-1}$ para decidir qué porcentaje de información recordar. Posteriormente, se determina el nodo de entrada $\tilde{\mathbf{C}}_t$ como una información candidata a incorporar. El porcentaje de esta información que finalmente se añade viene determinado por el vector de la puerta de entrada $\mathbf{I}_t$, multiplicando ambos elemento a elemento. Finalmente, este producto se añade a la memoria a largo plazo mediante una suma elemento a elemento.

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t \quad (2.17)$$

La última puerta a discutir es la puerta de salida $\mathbf{O}_t$. Esta compuerta regula qué porcentaje de la memoria a largo plazo formará el estado oculto de la celda $\mathbf{H}_t$.

El estado oculto no solo influye en la memoria a largo plazo, sino que además se utiliza para realizar predicciones de ese instante exacto t . Por lo tanto, la puerta de salida tiene dos interpretaciones:

determina qué parte de información del estado actual de la celda enviar al siguiente paso de tiempo y además regula qué parte de la memoria a largo plazo es relevante en el momento actual.

$$\mathbf{O}_t = \sigma(W_o \cdot [\mathbf{H}_{t-1}, \mathbf{X}_t] + b_o) \quad (2.18)$$

La memoria a corto plazo $\mathbf{H}_t$ de la celda se obtiene aplicando la multiplicación elemento a elemento del vector de la puerta de salida $\mathbf{O}_t$ con la tangente hiperbólica del estado interno $\mathbf{C}_t$.

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t) \quad (2.19)$$

2.2.17 Detección de *outliers* con LSTM

El procedimiento para detectar valores atípicos mediante redes LSTM consiste en modelar el comportamiento típico del sistema para luego poder identificar las anomalías.

En concreto se siguen los siguientes pasos (Malhotra et al., 2015, pág. 90):

- 1) Primero se entrena la red LSTM con datos libres de anomalías. Esta es una de las mayores desventajas de utilizar LSTM para la detección de *outliers*, ya que serán los investigadores quienes manualmente deban cribar los datos.
- 2) Una vez se tiene el modelo entrenado, se generan varias predicciones para cada momento t de la serie. Estas predicciones se generan desde diferentes momentos atrás en el tiempo ($t-1, t-2, \dots$).
- 3) Para cada momento t , se construye un vector de error comparando las predicciones con las observaciones reales.
- 4) Por último, sobre un subconjunto de validación libre de anomalías se ajusta una distribución Gaussiana multivariante a los vectores de error, caracterizando así el comportamiento del error en datos libres de anomalías.

Una vez se conocen los parámetros de la distribución Gaussiana, los investigadores pueden calcular la probabilidad de que una observación sea anómala o no.

2.2.18 Redes Convolucionales (CNN)

Las redes convolucionales son un tipo de red neuronal especializada en el procesamiento de datos en forma de cuadrícula. La dimensión de esta rejilla puede variar dependiendo del formato de los datos, por ejemplo, una dimensión para series temporales y dos dimensiones para imágenes. El nombre de red convolucional viene dado por la operación matemática que utiliza, la convolución (Goodfellow et al., 2016, pág. 330).

Ventajas de la convolución

Los motivos de utilizar convoluciones son varios.

- 1) En primer lugar, el uso de convoluciones permite reducir el coste computacional y de memoria del modelo. Si utilizásemos una red neuronal convencional para procesar una imagen de 1024×1024 píxeles requeriríamos 2^{20} pesos en cada una de sus neuronas. En una capa de tan solo 2048 neuronas, ya contaríamos con 2^{31} parámetros. El problema radica en tomar todos los píxeles individuales para la salida, ya que habitualmente la zona local de cada píxel es redundante. La convolución permite tomar regiones locales, lo que reduce significativamente la cantidad de parámetros del modelo (Goodfellow et al., 2016, pág. 335).
- 2) Otra gran ventaja de la convolución es que permite el uso compartido de parámetros. En lugar de aprender un conjunto de pesos para cada región específica de la imagen, se aprenden núcleos que se deslizan y se utilizan en cada posición de la entrada. Esto reduce drásticamente los parámetros y mejora la eficiencia computacional del modelo (Goodfellow et al., 2016, págs. 335-338).
- 3) Gracias a la propiedad anterior, el modelo tiene «equivarianza a la traslación»: si la entrada se desplaza, la salida cambiará exactamente de la misma manera (Goodfellow et al., 2016, págs. 338-339). En series temporales significa que los patrones detectados son invariantes en el tiempo, reduciendo otra vez los cálculos de la red neuronal.
- 4) Por último, la convolución permite trabajar con datos de entrada de tamaño variable (Goodfellow et al., 2016, pág. 335). Esto resulta práctico en producción, debido a que se puede entrenar el modelo con una sección de la serie para luego utilizarla en secuencias masivas de datos.

Convolución de una dimensión

En el caso particular de este trabajo los datos tienen una sola dimensión (el tiempo). Además, al estar formados por observaciones x_t en momentos específicos t decimos que son datos discretos. Por lo tanto, se utilizarán convoluciones unidimensionales discretas matemáticamente descritas por la siguiente operación³:

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t - a) \quad (2.20)$$

Donde:

- x representa las observaciones de la serie temporal.

³En matemáticas, la convolución pura requiere que el núcleo se invierta antes de deslizarse por la serie de datos. Debido a esto, en la mayoría de librerías de *machine learning* se utiliza correlación cruzada en lugar de convolución; sin embargo, en la comunidad se le sigue llamando convolución.

- w es el núcleo o *kernel*. Consiste en un vector de pesos.
- s es el resultado de la operación. Se le llama mapa de características.

El proceso de la convolución consiste en «deslizar» el núcleo a lo largo de la secuencia. Para cada posición t , se multiplican los «vecinos» por los elementos del núcleo y se suman todos los resultados. La salida $s(t)$ se puede interpretar como un promedio ponderado del instante t .

Arquitectura de una CNN

Podemos dividir la arquitectura de la red en dos partes: la extracción de características y la clasificación o predicción con capas densas.

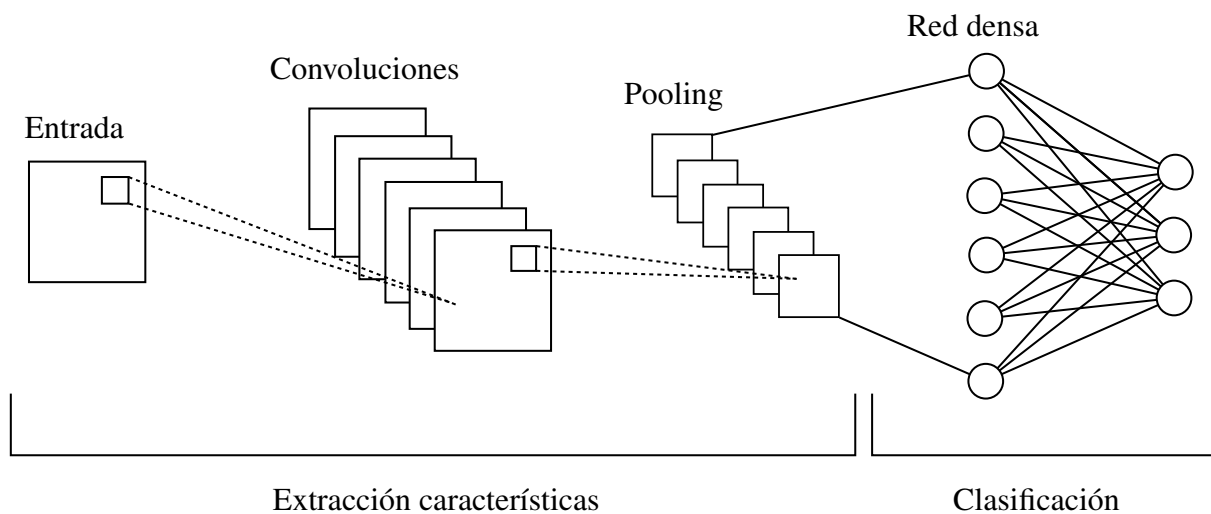


Figura 2.13: Arquitectura de una CNN.

En la etapa de extracción de características, se utiliza un bloque compuesto por tres capas especializadas: convolución, *pooling* y ReLU (Aggarwal, 2023, pág. 308). Este bloque se puede repetir varias veces para crear una red más profunda.

La primera parte del bloque consiste en una capa convolucional compuesta por uno o más núcleos a los que se denominan filtros. El número de filtros o canales de la capa convolucional se le llama profundidad (Aggarwal, 2023, pág. 307). Cada filtro tiene pesos diferentes y, por lo tanto, se dice que se especializa en un patrón de los datos. En imágenes, por ejemplo, podría centrarse en detectar los ojos de las personas.

La salida de cada filtro es un mapa de características. Para introducir no linealidad se le aplica a cada casilla del mapa de características una función de activación, siendo la más habitual la ReLU (Goodfellow et al., 2016, pág. 339).

Por último, se utiliza una capa de *pooling* para reducir la dimensionalidad de los datos. Esto alivia la carga computacional y además hace la red más resiliente ante traslaciones, es decir, los resultados

no cambian por mover la imagen. Existen varias funciones de pooling, una de ellas es la de Max Pooling. Esta función toma una región rectangular, por ejemplo de 3×3 , y la desliza por los datos. Para cada región que abarca la ventana, la función devuelve el valor máximo. Dependiendo del paso que elijamos (cuántos píxeles se mueve la ventana), los datos se reducirán calculando las nuevas dimensiones mediante la ecuación 2.21 (Aggarwal, 2023, pág. 315).

$$\begin{aligned} L_{nueva} &= \left\lfloor \frac{L_q - P_q}{S_q} \right\rfloor + 1 \\ B_{nueva} &= \left\lfloor \frac{B_q - P_q}{S_q} \right\rfloor + 1 \end{aligned} \quad (2.21)$$

Donde:

- L_q : es la longitud (largo) de la matriz o mapa de características de entrada.
- B_q : es la anchura (*breadth*) del mapa de características de entrada.
- P_q : es el tamaño espacial de la ventana de *pooling* (el filtro).
- S_q : es el tamaño del paso (*stride*) con el que se desliza la ventana.
- $\lfloor \dots \rfloor$: indica la operación matemática de truncamiento (redondear hacia abajo al número entero más cercano).

Por ejemplo, con una matriz 50×50 y un paso de 3 (sin solapamiento), la dimensión se reduce a 16×16 . Con un paso de 2 a 24×24 y con un paso de 1 a 48×48 .

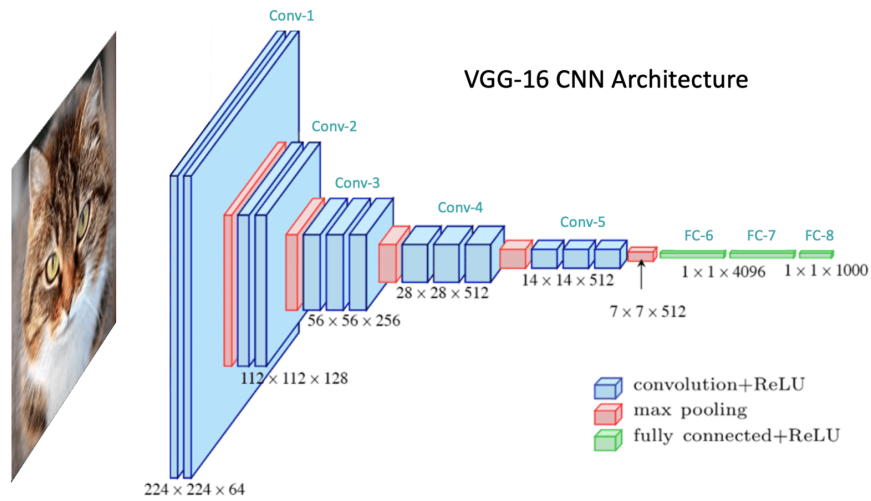


Figura 2.14: Arquitectura de VGG-16. Fuente: github.com/smallick/learnopencv.

Como se puede apreciar en la figura 2.14, una práctica habitual en las CNN profundas es reducir a la mitad la dimensionalidad de los datos con la función de pooling, pero duplicar el número de filtros en el siguiente bloque.

Una vez terminada la extracción de características se procede a la etapa de clasificación o predicción. En el caso de datos bidimensionales como imágenes, antes de clasificar se deben aplanar los datos del mapa de características con una capa de *flatten*. Esto se debe a que las redes neuronales tradicionales tienen por entrada un vector unidimensional. Después se clasifica o predice (en función de la tarea) con una red neuronal completamente conectada o densa como la de la figura 2.8.

2.2.19 DeepAnT

DeepAnT es una metodología para la detección de valores atípicos en series temporales basada en redes convolucionales. Según sus investigadores, este algoritmo supera el desempeño de otras técnicas desarrolladas por empresas como Yahoo, Twitter o Numenta. Para respaldar esta afirmación, los autores evaluaron 15 métodos del estado del arte sobre 10 conjuntos de datos (con un total de 433 series temporales), obteniendo DeepAnT los mejores resultados en la mayoría de los casos (Munir et al., 2019, pág. 1992). Además, destaca por ser un algoritmo no supervisado, lo que permite entrenar el modelo sin necesidad de eliminar o etiquetar previamente las observaciones anómalas; una característica que lo hace idóneo para entornos de producción reales. Por último, gracias al uso compartido de parámetros propio de las arquitecturas CNN, DeepAnT presenta la ventaja adicional de no requerir grandes volúmenes de datos históricos para su entrenamiento (Munir et al., 2019, resumen).

La arquitectura de DeepAnT se separa en dos módulos (Munir et al., 2019, pág. 1994). El primer módulo (*Time Series Predictor*) predice los siguientes valores de la serie temporal utilizando redes convolucionales. El segundo módulo (*Anomaly Detector*) se encarga de etiquetar un punto como anómalo utilizando el predictor.

Módulo predictor

El módulo de predicción consta de una CNN de dos capas convolucionales, dos capas de *max pooling* y una capa totalmente conectada.

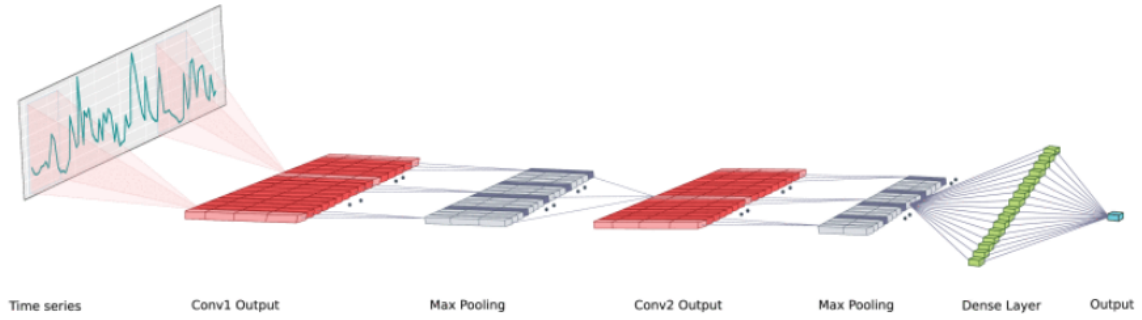


Figura 2.15: Arquitectura de DeepAnT. Fuente: Munir et al., 2019.

Como ya se ha comentado, el entrenamiento comienza con la extracción de características. Para ello, se utiliza la siguiente configuración (Munir et al., 2019, pág. 1996):

- Capas convolucionales de 32 filtros de dimensión 3×3 .
- Función de activación ReLU.
- Max Pooling de dimensión 2×2 .
- Función de pérdida MAE.
- Optimizador SGD.

Un detalle del entrenamiento de CNN en series temporales es el uso de la «ventana histórica» (Munir et al., 2019, pág. 1995). En vez de entrenar el modelo con todos los datos, estos se «trocean» en ventanas de tamaño w . En concreto, cuando la red va a predecir un valor x_t se toman los valores $\{x_{t-w}, \dots, x_{t-2}, x_{t-1}\}$ como contexto. El tamaño de la ventana histórica w se considera un nuevo hiperparámetro del modelo que afecta a la calidad de las predicciones.

Módulo detector

Con las predicciones del módulo anterior se realiza la categorización de las observaciones. El procedimiento consiste en dar a cada observación una puntuación anómala (*anomaly score*) utilizando la fórmula de la distancia euclidiana (Munir et al., 2019, pág. 1996):

$$(y_t, y'_t) = \sqrt{(y_t - y'_t)^2} \quad (2.22)$$

Donde:

- (y_t, y'_t) es el *anomaly score*.
- y_t es la observación que se quiere clasificar.
- y'_t es la predicción generada por la red neuronal.

Para determinar si una observación es un *outlier* o no, se computa esta puntuación y se compara con un umbral límite. Si la distancia supera dicho umbral, el punto se etiquetará como atípico. Este umbral es definido por los investigadores en función del caso particular.

2.3 Trabajos relacionados

En este apartado se presentan trabajos relacionados con la detección de *outliers* en el ámbito del tráfico de red. En primer lugar, se expone una comparativa entre un modelo estadístico clásico y uno basado en *machine learning*. Posteriormente, se describen diversas investigaciones centradas en el desarrollo de modelos híbridos que integran las técnicas discutidas previamente en este trabajo.

2.3.1 A Comparison of ARIMA and LSTM in Forecasting Time Series

En este trabajo, Siarni-Namini et al. (2018) llevan a cabo un análisis comparativo entre el modelo estadístico tradicional ARIMA y la red neuronal de memoria a largo plazo (LSTM) para el pronóstico de series temporales. Para validar su desempeño, los autores evaluaron ambos modelos utilizando 12 conjuntos de datos con una periodicidad mensual, cuyos registros abarcan el periodo comprendido entre 1985 y 2018.

En el estudio concluyeron que los modelos basados en deep learning eran superiores que los algoritmos tradicionales. En concreto, observaron una mejora media del 85 % en las predicciones de LSTM frente ARIMA.

Aunque el análisis se oriente hacia la predicción de series temporales, en este trabajo ya se ha expuesto el proceso de detección de anomalías fundamentado en los errores de predicción de los modelos LSTM y ARIMA.

2.3.2 Web traffic anomaly detection using C-LSTM neural networks

En este trabajo, Kim y Cho (2018) presentan una metodología para la detección de anomalías en el tráfico web mediante el desarrollo de un modelo híbrido denominado C-LSTM. Esta arquitectura integra CNN para la extracción de características espaciales y LSTM para el modelado de dependencias temporales.

El desempeño del modelo fue evaluado frente a diversas técnicas de *machine learning* utilizando el conjunto de datos *Webscope S5* de Yahoo y aplicando una validación cruzada de 10 pliegues. El modelo C-LSTM obtuvo los mejores resultados, superando a algoritmos convencionales como *Isolation Forest* y KNN. Asimismo, la investigación demostró que la combinación híbrida mejora significativamente la precisión frente al uso de las arquitecturas CNN y LSTM de forma aislada.

Finalmente, los autores añadieron una red neuronal profunda (DNN) para optimizar la separabilidad de los datos. Con la arquitectura completa (CNN + LSTM + DNN), los investigadores reportaron un *accuracy* del 98,6 % usando el mismo *dataset*.

2.3.3 Time-Series Anomaly Detection Service at Microsoft

En este artículo, Ren et al. (2019) desarrollan un de detección de anomalías en series temporales para los servicios de Microsoft. El sistema monitorea las métricas empresariales más relevantes, como los ingresos o las visualizaciones, en tiempo real.

El modelo utiliza un modelo adaptado de residuos espectrales (SR), una técnica que tradicionalmente se utilizaba en el ámbito de la visión por computador para detectar las zonas que más destacan en una imagen. En el trabajo, se utilizó el modelo para detectar las partes que más destacaban de los datos. Luego, una red CNN decidía si la observación era anómala o no.

Una de las principales ventajas de esta arquitectura SR-CNN es su naturaleza completamente no supervisada, una característica indispensable cuando se trabaja con los volúmenes de datos de empresas como Microsoft. Tras ser evaluado en entornos con datos de producción reales, el modelo híbrido demostró no solo una alta eficiencia computacional, sino también una gran capacidad de generalización ante diversos tipos de series temporales.

Capítulo 3

Aspectos metodológicos

En este capítulo se justifican las decisiones metodológicas y técnicas adoptadas durante el desarrollo del trabajo. En primer lugar, se describe la metodología seguida, detallando las fases del proyecto, los modelos elegidos para abordar la detección de anomalías y los criterios que motivaron su selección. A continuación, se enumeran las tecnologías y bibliotecas empleadas para la implementación, junto con los recursos computacionales utilizados durante el entrenamiento y la evaluación de los modelos.

3.1 Metodología

Para estructurar el desarrollo de este trabajo y cumplir con los objetivos planteados, se ha definido una metodología secuencial dividida en las siguientes fases.

3.1.1 Investigación del problema

Esta primera fase se centró en la exploración teórica y revisión de la literatura referente a las metodologías de detección de anomalías en series temporales. Mediante el análisis de las técnicas existentes, se logró establecer un marco de referencia sólido que permitió justificar y delimitar la elección de los modelos predictivos más pertinentes para abordar las particularidades del caso de uso.

3.1.2 Análisis exploratorio y preprocesamiento de datos

Una vez cargado el conjunto de datos, se realizó un breve preprocesamiento de los mismos con el fin de poder manipularlos adecuadamente.

Después se efectuó un análisis exploratorio de los datos. Para ello, se generaron diversas gráficas con el fin de identificar patrones subyacentes a simple vista, como pueden ser estacionalidad, ciclos o tendencias.

3.1.3 Etiquetado manual de incidencias

Para poder evaluar de forma objetiva los modelos de detección, fue necesario disponer de un conjunto de incidencias etiquetadas. Como el conjunto de datos no contaba con dichas etiquetas, se procedió a marcarlas de forma manual.

3.1.4 Diseño e implementación de modelos predictivos

Una vez completado el etiquetado, se procedió a la selección y entrenamiento de los modelos. Como ya se ha explicado anteriormente, la detección de valores extremos se fundamentará en los pronósticos de estos modelos.

En esta fase, se implementaron metodologías de distinta naturaleza matemática para abarcar diversos enfoques de aprendizaje. Como criterio fundamental para su selección, se estableció que los modelos debían permitir la integración de variables exógenas. Esta capacidad resultó indispensable para poder incorporar la información correspondiente a los días festivos, cuyo impacto altera el comportamiento habitual de la serie temporal.

Adicionalmente, antes de abordar los modelos más sofisticados, se incluyó un modelo base de referencia con el fin de comparar el grado de mejora aportado por el resto de técnicas.

En concreto, se seleccionaron los siguientes modelos:

Modelo base

Se implementó un detector inspirado en el que ya empleaba la empresa. Este se fundamenta en la media del valor observado a la misma hora durante los cinco días equivalentes anteriores, complementada con un conjunto de reglas que distinguen entre caídas totales y parciales. La sencillez del detector permitió evaluar de forma objetiva el grado de mejora de los algoritmos más complejos.

SARIMAX

La inclusión de un modelo perteneciente a la familia ARIMA se justifica por su relevancia histórica y su consolidación como el estándar metodológico clásico en la predicción de series temporales. Se optó por la variante SARIMAX para poder capturar los patrones estacionales inherentes al tráfico de datos y, además, para añadir la información de los días festivos mediante variables exógenas.

LSTM

Se seleccionó esta arquitectura de red neuronal recurrente debido a su capacidad para procesar datos secuenciales y capturar dependencias temporales a largo plazo, una característica esencial

para modelar los patrones complejos presentes en el tráfico de red. Además, como ya se explicó en el estado del arte, en la literatura hay evidencias de que los modelos de *deep learning* superan a los algoritmos tradicionales como ARIMA (Siami-Namini et al., 2018). Adicionalmente, esta implementación también permite la integración de variables exógenas.

CNN

Finalmente, se incorporó una red convolucional unidimensional. La elección de este modelo se fundamenta en dos características que son idóneas para este trabajo concreto.

En primer lugar, destaca su alta eficiencia computacional; las CNN resultan significativamente más rápidas en las fases de entrenamiento y predicción que las redes recurrentes. Esta agilidad de procesamiento es una gran ventaja para el despliegue del sistema de detección de anomalías en tiempo real propuesto en este trabajo.

En segundo lugar, tal y como se evidencia en la literatura mediante arquitecturas como *DeepAnT* (Munir et al., 2019), las redes convolucionales requieren de menos volúmenes de datos durante la fase de entrenamiento, manteniendo un alto rendimiento sin necesidad de conjuntos de datos masivos.

3.1.5 Ajuste de hiperparámetros

Una vez implementados los modelos, se procedió a la optimización de sus hiperparámetros. Para ello, se empleó un procedimiento automatizado de búsqueda que evalúa distintas configuraciones y selecciona aquella que maximiza el rendimiento sobre el conjunto de validación.

3.1.6 Evaluación y análisis comparativo

A continuación, se evaluaron y compararon los diferentes modelos según su capacidad para detectar las incidencias previamente etiquetadas. Se utilizaron dos métricas diferentes para cuantificar el rendimiento. La primera es una métrica propia basada en eventos, diseñada específicamente para este caso de uso. La segunda es la puntuación NAB (Ahmad et al., 2017), ampliamente consolidada en la literatura de detección de anomalías en series temporales.

3.1.7 Despliegue para detección de anomalías en tiempo real

Finalmente, se desarrolló una plataforma para la detección y visualización de anomalías en tiempo real. El objetivo de esta última etapa es prototipar una posible solución real al problema de la empresa.

3.2 Tecnologías empleadas

3.2.1 Python 3.11.8

Se ha seleccionado Python como lenguaje de programación para este trabajo. Esta decisión se fundamenta en su versatilidad, su rapidez de desarrollo y, sobre todo, en el amplio ecosistema de bibliotecas disponibles para el análisis de datos y el *machine learning*.

Las librerías de Python que se han utilizado en el desarrollo son las siguientes:

NumPy 2.4.4

Librería que fundamenta varias de las bibliotecas de análisis de datos. Se ha utilizado para realizar transformaciones matemáticas a los datos.

Pandas 3.0.3

Pandas es la biblioteca principal para la manipulación y el análisis de datos estructurados. En este trabajo se ha utilizado para cargar el conjunto de datos y para el preprocesamiento del mismo.

Matplotlib 3.10.9

Módulo de referencia para la generación de gráficas en Python. Se ha empleado en el análisis exploratorio de los datos (EDA) y en la creación de las figuras incluidas en este documento.

Plotly 6.7.0

Biblioteca de visualización interactiva para Python. En este trabajo se ha utilizado para construir el visor con el que se etiquetaron manualmente las incidencias del conjunto de datos.

Statsmodels 0.14.6

Biblioteca especializada en el análisis estadístico dentro del ecosistema de Python. En esta investigación, se ha empleado para la ejecución de pruebas de contraste de hipótesis y para la construcción y el ajuste de los modelos predictivos ARIMA.

Statsforecast 2.0.2

Biblioteca especializada en modelos clásicos de series temporales. En este trabajo se ha empleado para el ajuste automático del modelo SARIMAX mediante su algoritmo `AutoARIMA`.

Scikit-learn 1.8.0

Biblioteca de referencia para tareas generales de *machine learning* en Python. Se ha utilizado para el escalado de las variables de entrada de las redes neuronales mediante `MinMaxScaler`.

Keras 3.14.1

TensorFlow es una biblioteca fundamental de código abierto orientada a la creación de modelos de aprendizaje automático. Keras actúa como su Interfaz de Programación de Aplicaciones (API) de alto nivel, facilitando su uso. En este trabajo, se ha empleado para ensamblar y entrenar las arquitecturas de redes neuronales convolucionales (CNN) y recurrentes (LSTM) destinadas a la detección de anomalías.

Optuna 4.8.0

Biblioteca de optimización de hiperparámetros mediante búsqueda automatizada. Se ha empleado para encontrar la configuración que maximiza el F_1 de detección de los modelos DeepAnT y LSTM, almacenando los resultados en una base de datos SQLite.

Holidays 0.96

Biblioteca que recopila los días festivos oficiales de un gran número de países. Se ha empleado para construir la variable exógena `festivo` a partir del calendario nacional español.

Dash 4.1.0

Framework de aplicaciones web ligeras en Python, construido sobre Plotly. Se ha empleado para implementar el cuadro de mando de la aplicación de detección de anomalías en tiempo real.

3.2.2 Jupyter Notebook

El desarrollo del código se ha realizado en *notebooks* de Jupyter. Esta herramienta permite documentar el proyecto mediante celdas de Markdown. Además, agiliza el desarrollo al evitar ejecutar todo el proyecto cada vez que se realiza un cambio. Esto lo consigue separando el código en celdas independientes que se pueden ejecutar de forma individual. Otra ventaja de este entorno es que guarda los resultados de la ejecución; esto permite compartir el documento sin necesidad de que el receptor tenga que volver a ejecutarlo.

3.2.3 Recursos computacionales

El entrenamiento de los modelos propuestos se ha llevado a cabo de manera local. En concreto, el equipo físico cuenta con una tarjeta gráfica NVIDIA GeForce RTX 2070. El uso de este *hardware* dedicado ha resultado fundamental para acelerar significativamente el proceso de entrenamiento de las redes neuronales profundas.

Capítulo 4

Desarrollo del trabajo

En este capítulo se muestra el desarrollo del trabajo. Primero, se documenta el preprocesamiento del conjunto de datos y el etiquetado manual de las incidencias, seguido del análisis exploratorio de la serie de tráfico. A continuación, se define el marco común de evaluación y se implementan los cuatro detectores propuestos. El capítulo concluye con el ajuste de hiperparámetros, la comparativa de resultados y la descripción del cuadro de mando de monitorización en tiempo real.

4.1 Preparación y preprocesamiento de los datos

Se utilizaron datos de producción reales de los años 2023, 2024 y 2025 de una aplicación que da servicio a nivel nacional. La empresa con la que se ha colaborado en este trabajo desea permanecer anónima, por lo que se omitirán los detalles sobre esta aplicación. Lo único a mencionar es que tiene servidores en varias de las provincias de España.

Tabla 4.1: Estructura del conjunto de datos.

Atributo	Tipo de dato	Descripción
Operación	Cadena de texto	Tipo de petición (consulta o descarga).
Fecha-hora	Marca de tiempo	Fecha y hora exacta en la que se registra la operación (formato YYYY-MM-DD HH:MM).
Métrica	Entero	Número de peticiones por minuto recibidas por la aplicación.

Como se observa en la tabla 4.1, cada observación dispone de 4 columnas. La columna *Operación* indica que se trataba de una petición de consulta o de descarga. Para esta investigación se usarán solo las peticiones de tipo consultas debido a que son más abundantes.

El atributo *Métrica* contiene el número de peticiones que se recibieron a nivel nacional en el último minuto. A pesar de tener registros cada minuto, para el desarrollo de este trabajo se redimensionó

la serie temporal a una periodicidad de 5 minutos. El motivo de esta redimensión es agilizar los entrenamientos de los modelos y aliviar la carga de memoria de los equipos. Además, se observó mejor rendimiento que con *resample* de un minuto, posiblemente porque suavizaba el ruido de la serie.

El conjunto de datos original estaba dividido en 1096 archivos en formato CSV. La primera operación consistió en unir todas sus filas en un *dataframe* de Pandas. Después, se procesó el campo *Fecha-hora* para que tuviese el tipo de variable correcta. Además, se filtró la serie para solo trabajar con las consultas.

En la figura 4.1 se muestra la serie temporal resultante una vez aplicadas las operaciones anteriores. Se observa la marcada estacionalidad diaria y semanal del tráfico, así como un incremento del volumen de peticiones en octubre de 2025.

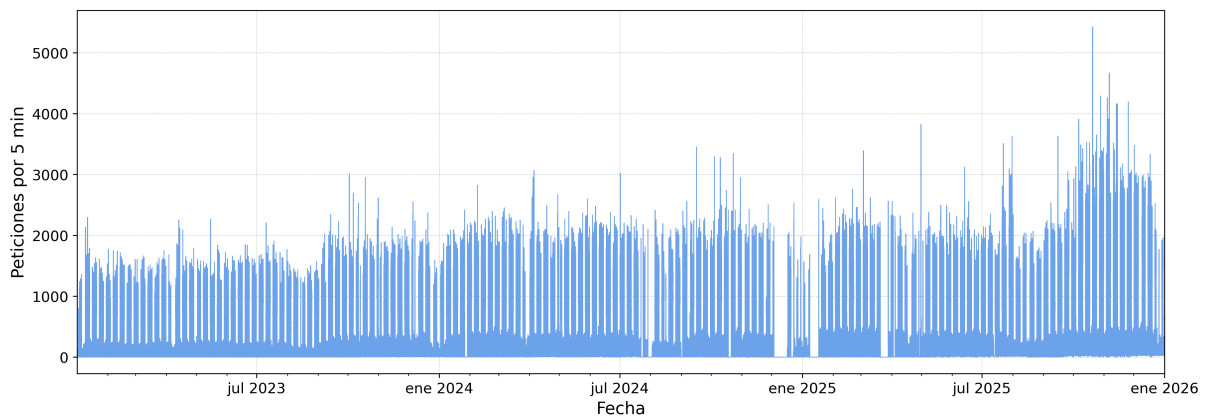


Figura 4.1: Serie temporal de los datos.

4.1.1 Etiquetación de *outliers*

Es indispensable conocer las anomalías en los datos para poder construir y validar los modelos de detección. Sin embargo, al tratarse de datos reales, las observaciones no estaban etiquetadas. El único registro que se obtuvo fue una conversación de mensajería en la que se reportaban las incidencias de los servidores. De este *chat* se obtuvo una tabla estructurada utilizando expresiones regulares.

Tabla 4.2: Estructura de las incidencias procesadas.

Atributo	Tipo de dato	Descripción
Inicio	Marca de tiempo	Fecha y hora de comienzo del evento (YYYY-MM-DD HH:MM:SS).
Fin	Marca de tiempo	Fecha y hora de finalización del evento.
Duracion_horas	Decimal	Tiempo total transcurrido expresado en horas (ej. 1.50).
Titulo	Cadena de texto	Título de la incidencia.
Descripcion	Cadena de texto	Detalle extendido del problema.
Num_mensajes	Entero	Cantidad total de mensajes asociada a la incidencia.
Estado	Cadena de texto	Situación actual de la incidencia.
Autores	Cadena de texto	Autor del mensaje.

En total se almacenaron 21 anomalías asociadas a caídas de la aplicación durante el periodo 2023–2025.

4.1.2 Revisión manual de caídas

A pesar de tener constancia de estas 21 incidencias, la revisión de la serie temporal reveló que numerosas caídas del servicio no habían sido registradas. Por lo tanto, se tuvo que realizar un cribado manual exhaustivo de los datos.

Durante este proceso, las interrupciones se clasificaron en dos categorías: totales y parciales. Se consideró como caída total aquella en la que el flujo de peticiones descendió a cero, mientras que las parciales se definieron como aquellas que presentaban una reducción drástica del tráfico sin llegar a la interrupción completa.

En la figura 4.2 se ilustran ambos comportamientos sobre la serie real de consultas: en el caso superior, el tráfico desciende abruptamente hasta cero durante el intervalo de la incidencia, mientras que en el inferior se aprecia una degradación del volumen de peticiones que, sin embargo, no llega a anularse por completo.

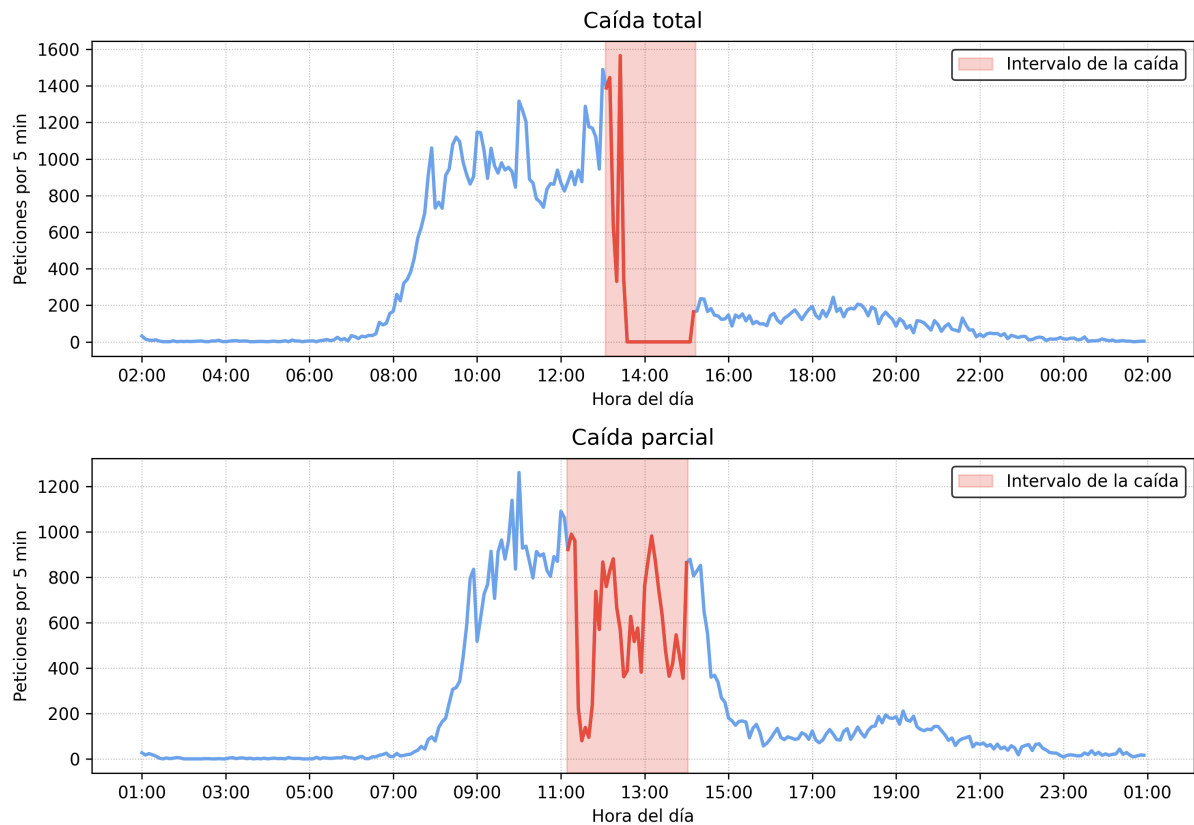


Figura 4.2: Ejemplos de caída total y caída parcial.

La validación y categorización de las anomalías requirió la implementación de un visor a medida. Se construyó una interfaz basada en Plotly con las siguientes características:

- Renderizado de la serie temporal.
- Visualización de las incidencias obtenidas del chat.
- Creación de franjas de anomalías.
- Comparación con el tráfico estimado por el modelo base.

En la figura 4.3 se muestra la interfaz web del visor de incidencias. El programa dibuja tanto el dato real como el estimado. Además, marca los posibles valores atípicos con cruces (caídas totales) o triángulos (caídas parciales). Las franjas rojas y azules delimitan los *outliers* marcados por los investigadores.



Figura 4.3: Visor de incidencias.

Mediante el uso de esta herramienta, se anotaron 355 caídas. Como se observa en la figura 4.4, la mayoría de estas interrupciones son de caídas totales.

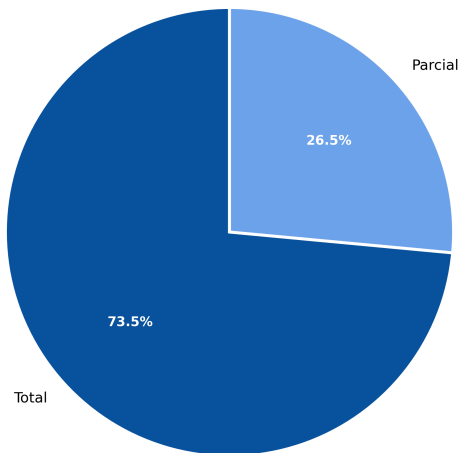


Figura 4.4: Distribución del tipo de caídas.

Las caídas totales son fácilmente identificables, ya que es muy improbable que en un horario laboral el sistema llegue al cero absoluto. Como se verá en el siguiente apartado, las interrupciones parciales dan lugar a más dudas y no siempre se puede determinar la causa de su comportamiento.

4.1.3 Construcción de variables exógenas

La naturaleza del tráfico de peticiones está fuertemente condicionada por factores externos que ninguno de los modelos puede inferir por sí solo. Por ejemplo, en la figura 4.5 se comparan dos miércoles laborables con uno festivo.

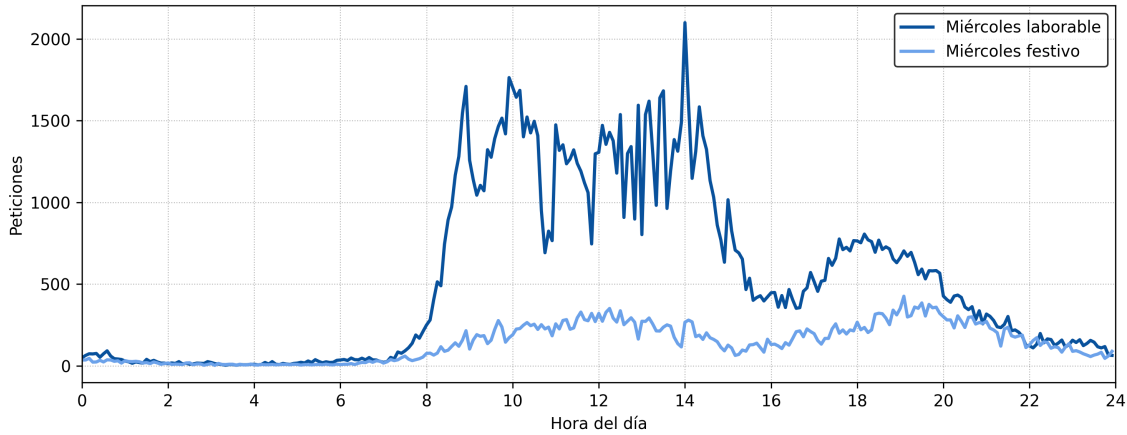


Figura 4.5: Comparación del volumen de peticiones de consulta entre un miércoles laborable y un miércoles festivo.

Para hacer frente a este problema, en este trabajo se optaron por modelos que admiten variables exógenas. En concreto, se construyeron tres variables exógenas:

En primer lugar, se añadió el indicador booleano `festivo` que denota cuando un día es festivo nacional. Para su construcción se empleó la biblioteca `holidays`.

Después, se modelaron el día de la semana y la hora del día. En lugar de utilizar los rangos habituales de horas (0-23) y de días de la semana (0-6), se transformaron mediante senos y cosenos. El motivo de aplicar estas funciones es evitar saltos abruptos: si no transformamos las horas, el modelo puede entender que el domingo (día 6) está lejos del lunes (día 0).

$$\text{dow_sin} = \sin\left(\frac{2\pi \cdot d}{7}\right), \quad \text{dow_cos} = \cos\left(\frac{2\pi \cdot d}{7}\right) \quad (4.1)$$

Donde d es el día de la semana representado como un valor entero entre 0 (lunes) y 6 (domingo).

De forma análoga, para la hora del día se utilizó la siguiente codificación con un período de 1440 minutos:

$$\text{time_sin} = \sin\left(\frac{2\pi \cdot m}{1440}\right), \quad \text{time_cos} = \cos\left(\frac{2\pi \cdot m}{1440}\right) \quad (4.2)$$

Siendo m el número de minutos transcurridos desde la medianoche.

Tras aplicar todas las transformaciones anteriores, el conjunto de datos definitivo presenta la estructura recogida en la tabla 4.3.

Tabla 4.3: Estructura del conjunto de datos tras el preprocesamiento.

Atributo	Tipo de dato	Descripción
Fecha-hora	Marca de tiempo	Índice temporal con periodicidad de 5 minutos.
y	Entero	Número total de peticiones de consulta agregadas en el intervalo de 5 minutos.
festivo	Booleano	Indicador que toma el valor 1 si el día es festivo nacional español (o 24/31 de diciembre) y 0 en caso contrario.
dow_sin	Decimal	Componente seno de la codificación cíclica del día de la semana.
dow_cos	Decimal	Componente coseno de la codificación cíclica del día de la semana.
time_sin	Decimal	Componente seno de la codificación cíclica de la hora del día.
time_cos	Decimal	Componente coseno de la codificación cíclica de la hora del día.

4.1.4 División del conjunto de datos

Tras el preprocesamiento, se dividió el conjunto de datos en tres subconjuntos: entrenamiento, validación y test. Al tratarse de una serie temporal, la división debe realizarse de forma cronológica, ya que tomar muestras aleatorias rompería la continuidad temporal de la serie y confundiría a los modelos:

- El primer 40 % de la serie se reservó para el entrenamiento.
- El siguiente 10 % se destinó a la validación.
- El 50 % final de la serie se utilizó como conjunto de test sobre el que se evaluó el desempeño de todos los modelos.

En la figura 4.6 se muestra el resultado de esta partición sobre la serie completa, junto con la localización temporal de las incidencias anotadas. Como se ve, sin quererlo, la mayoría de las incidencias quedan fuera del conjunto de entrenamiento. Esto resultará beneficioso para el entrenamiento de los modelos.

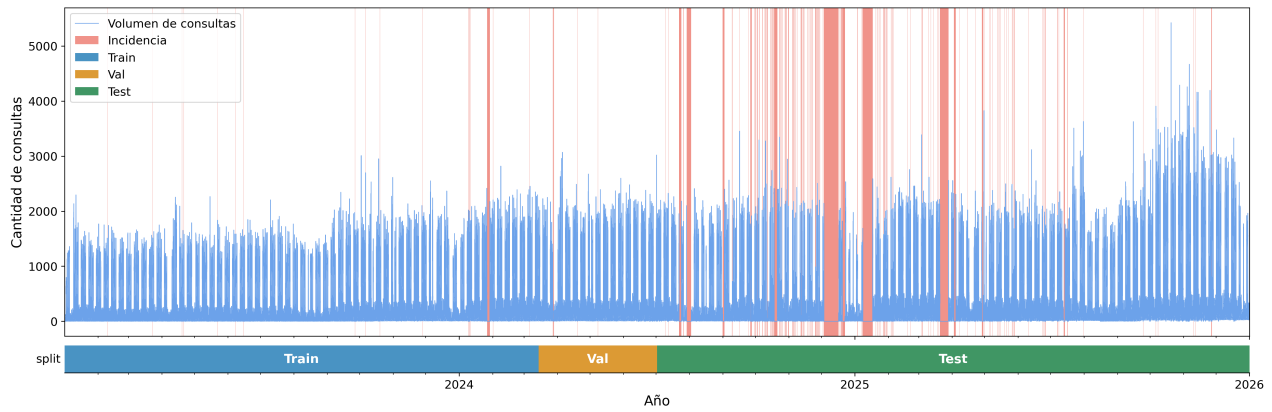


Figura 4.6: División cronológica del conjunto de datos en entrenamiento, validación y test. Las franjas rojas verticales indican las incidencias anotadas.

4.2 Análisis exploratorio de los datos

Una vez preprocesado del conjunto de datos, se procedió a un análisis exploratorio con el fin de descubrir la naturaleza y patrones de la serie temporal.

4.2.1 Patrón diario

La mayoría de días laborables transcurren de un modo similar con un patrón fácilmente reconocible. El comportamiento del tráfico durante un día laborable típico responde a un patrón fácilmente reconocible. La figura 4.7 lo ilustra sobre una jornada representativa y sin incidencias registradas.



Figura 4.7: Patrón diario habitual del tráfico de consultas.

La figura 4.7 muestra un día de la serie en el que se aprecian bien estos patrones. En primer lugar, el tráfico es casi nulo hasta las seis de la mañana, en donde va creciendo hasta alcanzar su pico, habitualmente sobre las 9:00. Entonces, en la mayoría de casos, ocurre una breve caída en

las peticiones. A continuación, el tráfico se mantiene nivelado con oscilaciones en el volumen de consultas. Entonces, a las 14:00 suele producirse otra caída abrupta para luego tener un repunte asociado a la gente que vuelve de comer.

Cabe destacar que ambas caídas no se asocian al mal funcionamiento del sistema sino al comportamiento inherente a los datos ¹.

Tras este último pico el tráfico decae para luego estabilizarse, trazando un pequeño «monte» entre las 16:00 y las 22:00.

4.2.2 Mantenimientos del sistema

Un fenómeno recurrente en el conjunto de datos son los mantenimientos programados. Estos se inician al final de la jornada del viernes y se prolongan hasta la mañana del lunes siguiente. La figura 4.8 ilustra uno de estos episodios.

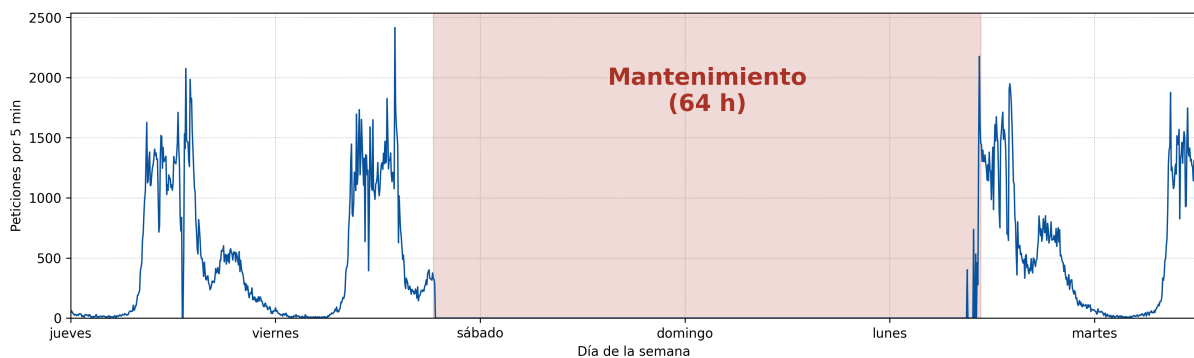


Figura 4.8: Mantenimiento programado de viernes a lunes.

Durante este tipo de episodios el tráfico cae a cero de manera abrupta y se mantiene en dicho valor durante aproximadamente 64 horas. Aunque se trata de un evento programado se sigue considerando una anomalía del sistema debido a su rareza.

4.2.3 Cambio estructural en 2025

En el año 2025 se observa una mayor variabilidad del dato y un incremento sustancial de incidencias. La figura 4.9 ilustra esta variabilidad sobre un día concreto. Además, como se aprecia en la figura 4.1, a finales de 2025 el volumen de peticiones sufre un aumento, el cual se podría considerar un *outlier* de cambio de nivel (LS).

¹Ambos patrones se tuvieron en cuenta a la hora de etiquetar los datos como anomalos

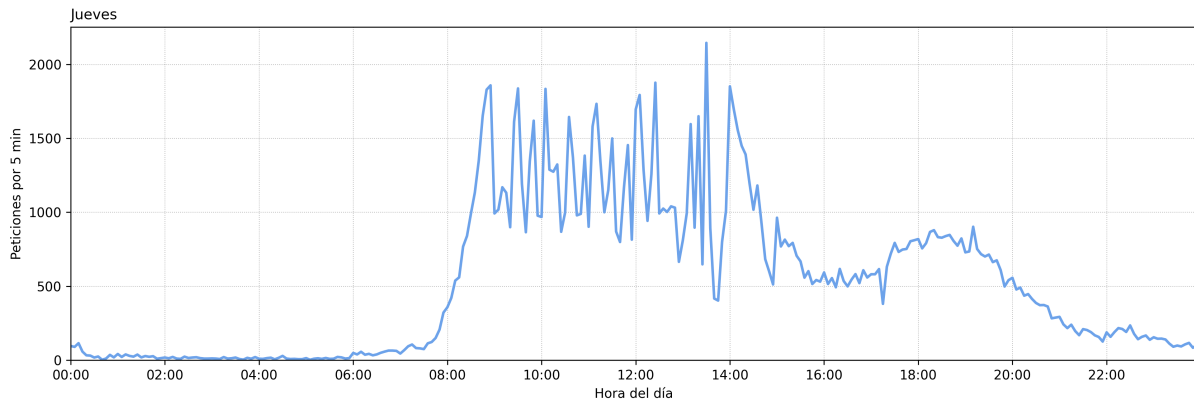


Figura 4.9: Tráfico de un día representativo posterior al cambio estructural (sin incidencias registradas).

4.2.4 Otros aspectos analizados

Además de los patrones ya comentados, se detectó un tráfico reducido en los días festivos y una leve estacionalidad semanal por la que cada día de la semana tenía pequeñas diferencias en su comportamiento.

4.3 Marco común de evaluación

Para determinar el rendimiento de los modelos primero hay que establecer que es un acierto y que es un fallo. En este trabajo, se optaron por dos métricas diferentes. En primer lugar, se diseñó una evaluación propia adaptada al caso de uso. Después, con el fin de contrastar esta nueva métrica, se utilizó la puntuación NAB propuesta por Ahmad et al. (2017), ampliamente utilizada en la literatura de detección de anomalías en series temporales. Esta sección describe ambas métricas, además del procedimiento del cálculo del umbral de detección.

4.3.1 Evaluación basada en eventos

Las métricas habituales de detección de anomalías evalúan cada instante de la serie de forma independiente, contabilizando aciertos y errores de forma discreta. Sin embargo, este planteamiento no refleja el comportamiento habitual de los sistemas de alarmas: El operador del sistema no marca cada instante con anómalo o normal, sino que alerta de caídas que se extienden por un periodo de tiempo. Por ejemplo, una incidencia que se prolonga durante una hora y resulta correctamente detectada en cinco de sus doce instantes constituye, desde el punto de vista operativo, un único acierto y no doce.

Por esta razón, se diseñó una métrica propia, denominada evaluación basada en eventos (*event-based*), que considera cada incidencia del siguiente modo:

- **Verdaderos positivos (TP)**: una incidencia real se contabiliza como acierto cuando al menos un instante dentro de su franja temporal ha sido marcado como anomalía por el modelo.
- **Falsos negativos (FN)**: incidencias reales para las que ningún instante interno ha sido marcado como anomalía.
- **Falsos positivos (FP)**: cada secuencia contigua de instantes marcados como anomalía que no solape con ninguna incidencia real se contabiliza como una única falsa alarma.

A partir de estos recuentos se obtienen la precisión 4.3, el *recall* 4.4 y la puntuación F_1 4.5.

$$\text{precisión} = \frac{TP}{TP + FP} \quad (4.3)$$

$$\text{recall} = \frac{TP}{TP + FN} \quad (4.4)$$

$$F_1 = 2 \cdot \frac{\text{precisión} \cdot \text{recall}}{\text{precisión} + \text{recall}} \quad (4.5)$$

Este modo de evaluar no tiene en cuenta cuanto se tarda en reportar las caídas. En consecuencia, se consideran dos magnitudes complementarias: *fp_minutos*, que mide el tiempo total de alarma falsa al que sería expuesto el operador, y *latencia_medio*, que mide el tiempo medio en el que se tarda en dar la alarma sobre una incidencia.

4.3.2 Métrica NAB de Numenta

Con el fin de comparar los modelos desarrollados con los de la literatura, se decidió evaluar los modelos con el *NAB score*, propuesto por Ahmad et al. (2017). Esta puntuación está muy consolidada en la investigación de detección de anomalías en series temporales y además fue ideada para datos en tiempo real como con los de este trabajo.

Se utilizó el código público de Numenta (Ahmad et al., 2019), el cual asume las anomalías como puntos discretos. Como las incidencias de este trabajo fueron marcadas mediante franjas, se tomó el primer punto de cada franja anómala.

A continuación se detalla el procedimiento para calcular la puntuación NAB:

- 1) En primer lugar, en torno a cada anomalía real se define una ventana temporal cuya longitud equivale al 10 % de la longitud total de la serie, dividida entre el número de anomalías totales en los datos (Ahmad et al., 2017, sec. 3.2). Si dos ventanas se solapan, se fusionan en una sola.

- 2) En segundo lugar, se puntúa cada observación tomando en cuenta su posición relativa a la ventana, de modo que cuanto más temprano se detecte más nota recibe. Sin embargo, solo se cuenta la primera detección; las demás son redundantes. Los falsos positivos restan a la puntuación, restando menos si se encuentran cerca de la ventana. Si una ventana no es detectada, también resta a la puntuación.

La puntuación se puede ajustar al caso de uso mediante el perfil de aplicación (Ahmad et al., 2017, sec. 3.2). Este perfil de aplicación está formado por parámetros que asignan una importancia específica a cada tipo de error. En este trabajo se emplea el perfil *standard*, que es el habitualmente utilizado en la literatura.

- 3) Por último, la puntuación se normaliza al intervalo $[0, 100]$ tomando como referencia un detector perfecto, que dispararía la primera alarma exactamente al inicio de cada incidencia sin generar falsos positivos, y un detector nulo, que no detectaría ninguna anomalía (Ahmad et al., 2017, sec. 4.2). De esta forma, el resultado es comparable entre conjuntos de datos de distinta longitud y número de incidencias.

4.3.3 Cálculo del umbral de detección

Todos los modelos basados en predicción (SARIMAX, DeepAnT y LSTM) utilizan el mismo sistema de detección. Para determinar si una observación es atípica, se predice el valor esperado por el modelo y se calcula su *anomaly score* (2.22).

Una observación se etiqueta como anómala cuando su *anomaly score* supera un determinado umbral. Este umbral afecta a la precisión y *recall* del modelo: un umbral bajo causa más falsos positivos, mientras que un valor alto deja sin detectar a la mayoría de incidencias reales.

Para poder encontrar un umbral adecuado, se diseñó un procedimiento de optimización basándose en el conjunto de validación². Concretamente, se emplea el método de Brent, implementado en `scipy.optimize.minimize_scalar`. El procedimiento explora el intervalo $[\min(s_t), \max(s_t)]$ y devuelve el valor que maximiza la métrica de evaluación sobre los *scores* de validación.

Como se trabaja con dos métricas diferentes (*event-based* y NAB), se genera un umbral optimizado para cada una.

Por último, se añade una restricción a la detección de anomalías: solo se marcan las observaciones cuyas predicciones son mayores que el valor observado. De este modo, los modelos solo detectan las caídas del sistema. Esta decisión fue tomada atendiendo al objetivo concreto de este trabajo, que es detectar las interrupciones del tráfico de peticiones.

²Optimizar el umbral utilizando observaciones del conjunto de test podría verse como poco honesto.

4.4 Modelos

4.4.1 Modelo base

Antes de entrenar los modelos complejos, se desarrolló un detector de referencia con el propósito de establecer una línea base de comparación. Su construcción es sencilla y se basa en el detector real que se empleaba en la aplicación de la empresa.

La predicción se obtiene calculando la media del valor observado en la misma hora en los últimos 5 días de la semana no festivos. Por ejemplo, si se quiere etiquetar una observación de un lunes a las 19:00 se computa el valor medio de los últimos 5 lunes a las 19:00. Una vez se tiene este valor medio se aplican las siguientes reglas de detección.

- Únicamente se evalúan los instantes que pertenecen a días laborables (de lunes a viernes), en horario comprendido entre las 7:00 y las 22:00 y que no coincidan con festivos nacionales.
- Si el pronóstico es inferior a un umbral mínimo (en este trabajo, 100 consultas), se considera que la franja carece de actividad relevante y se descarta.
- Se marca como caída total cualquier observación en la que el tráfico sea exactamente cero.
- Se marca como caída parcial cualquier observación en la que el tráfico observado sea inferior a un umbral obtenido de la forma previamente descrita.

4.4.2 Modelo SARIMAX

El segundo modelo desarrollado pertenece a la familia clásica de modelos ARIMA, descrita en el marco teórico. En concreto, se trata de un SARIMAX, que extiende el ARIMA con una componente estacional y con variables exógenas. La motivación de incluirlo en este trabajo es comparar un enfoque clásico con las técnicas modernas de *deep learning*.

Preparación de los datos

A diferencia del resto de modelos, el SARIMAX no se ajustó sobre la serie con periodicidad de 5 minutos. En su lugar, se agruparon los datos en periodos de 30 minutos. Se debe a una limitación técnica, ya que bajar la periodicidad dispara el uso de RAM y CPU durante el ajuste del modelo.

En cuanto a las variables exógenas, no se incorporaron las cinco construidas. Se descartaron las variables `time_sin` y `time_cos` porque la información que aportan ya es capturada por los parámetros estacionales del modelo.

Análisis de estacionariedad y estacionalidad

Para poder justificar el uso de SARIMA³, se realizó un análisis de la estacionalidad de los datos. Para ello, se descompuso la serie en tendencia, estacionalidad y residuo como se muestra en la figura 4.10.

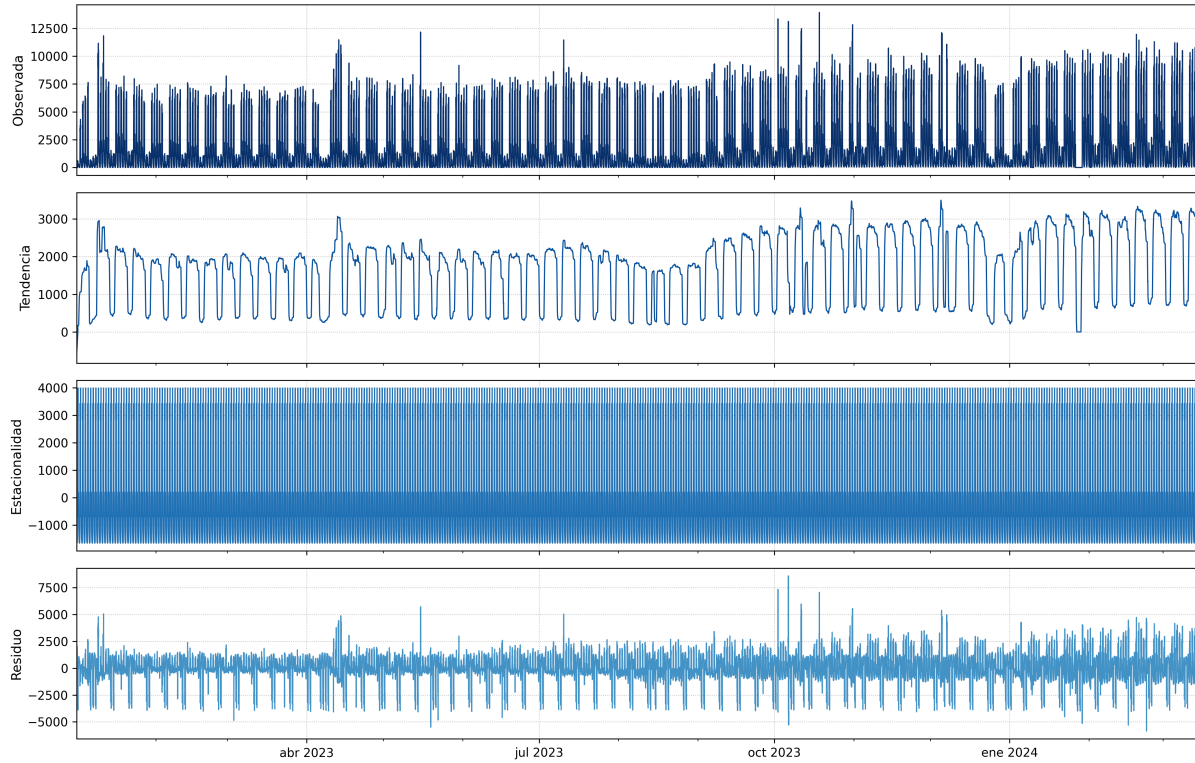


Figura 4.10: Descomposición estacional aditiva de la serie con período $m = 48$.

Al calcular la varianza atribuible a cada componente, se vio que aproximadamente el 60 % de la varianza observada quedaba explicada por la componente estacional, lo que permite hablar de una estacionalidad fuerte. Por otro lado, se inspeccionaron los correlogramas ACF y PACF de la serie hasta 72 retardos, recogidos en la figura 4.11.

³Si la serie no presentase estacionalidad, las componentes estacionales de SARIMA serían redundantes.

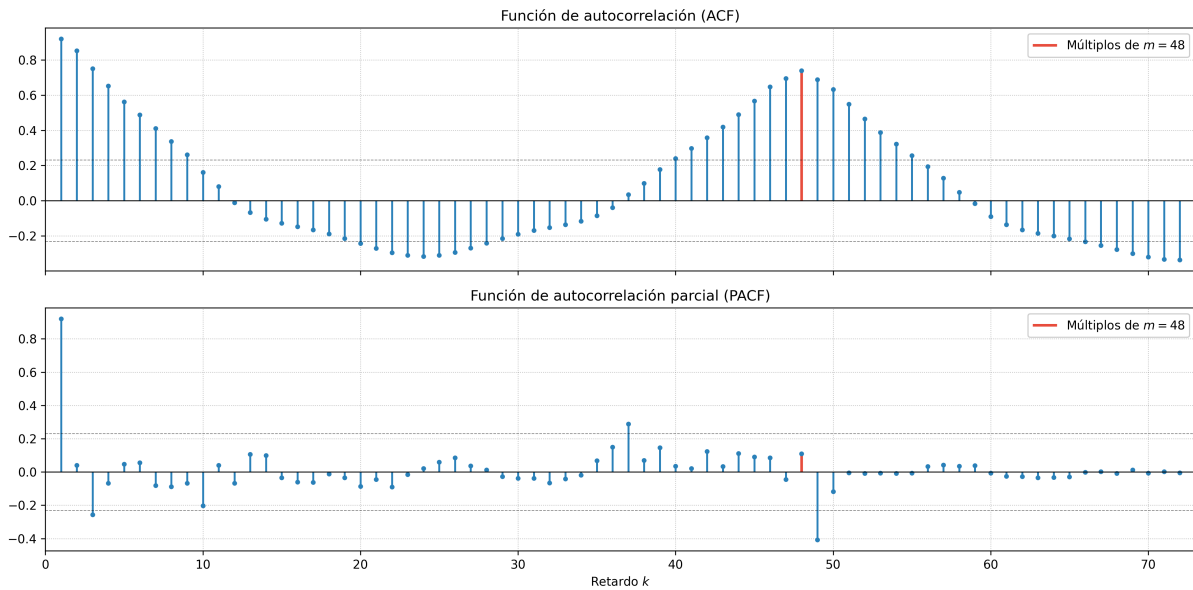


Figura 4.11: Correlogramas ACF y PACF de la serie hasta 72 retardos.

La aparición de picos pronunciados en los múltiplos del período diario confirma la estacionalidad diaria de la serie. Cabe resaltar que se observaron también picos en múltiplos de $7 \cdot 48 = 336$, los cuales indican una estacionalidad semanal adicional. Sin embargo, el modelo clásico no puede modelar ambas estacionalidades ⁴.

En cuanto a la estacionariedad, ya se observó en el análisis exploratorio cambios de tendencia y varianza. Por lo tanto, se diferenció la serie como se describe en el marco teórico (2.8). Tras esta transformación, se validó el resultado mediante contrastes de hipótesis. En concreto, se aplicaron el contraste ADF y el contraste KPSS, ambos para comprobar si una serie es estacionaria o no. Ambos contrastes indicaron que la serie era estacionaria tras la transformación.

Ajuste del modelo

La selección de los órdenes $(p, d, q) \times (P, D, Q)_m$ se realizó de forma automática mediante el algoritmo `AutoARIMA` de la biblioteca `statsforecast`. Dicho algoritmo constituye una adaptación a Python de la función `auto.arima()` de R, la cual implementa el procedimiento de Hyndman-Khandakar (Hyndman & Athanasopoulos, 2018, sec. 8.7).

El ajuste se realizó sobre el conjunto de validación y entrenamiento concatenado, con el fin de abarcar la mayor cantidad de datos históricos. De todos los modelos, SARIMAX resultó ser el más largo de entrenar debido a que el algoritmo se ejecuta en un solo procesador de la CPU.

⁴En su lugar, se puede recurrir a modelos de descomposición más modernos como Prophet de Meta.

Calidad de las predicciones

Tras el ajuste del modelo, se comprobó la calidad de sus pronósticos sobre el conjunto de test utilizando diferentes métricas. Adicionalmente, se evaluó el modelo utilizando el conjunto de test sin anomalías para dar una medida más fiel de su rendimiento predictivo.

Métrica	Test completo	Test sin incidencias
MAE	1889.43	1684.67
RMSE	2467.50	2165.06
MAPE	4.977	5.025
sMAPE	1.099	0.993
Sesgo	-1342,86	-1089,39
R^2	0.237	0.429

Tabla 4.4: Métricas de predicción de SARIMAX sobre el conjunto de test.

En la tabla 4.4 se muestra que el modelo apenas alcanza un coeficiente de determinación de 0.43 al descartar las anomalías. El sesgo, marcadamente negativo, indica que el modelo tiende a sobreestimar el valor real de forma sistemática.

Adicionalmente, en la figura 4.12 se representan los valores reales y los predichos sobre una semana representativa del conjunto de test. Se observa que el modelo no ha captado bien la variable exógena del día de la semana, ya que produce el mismo tráfico todos los días.

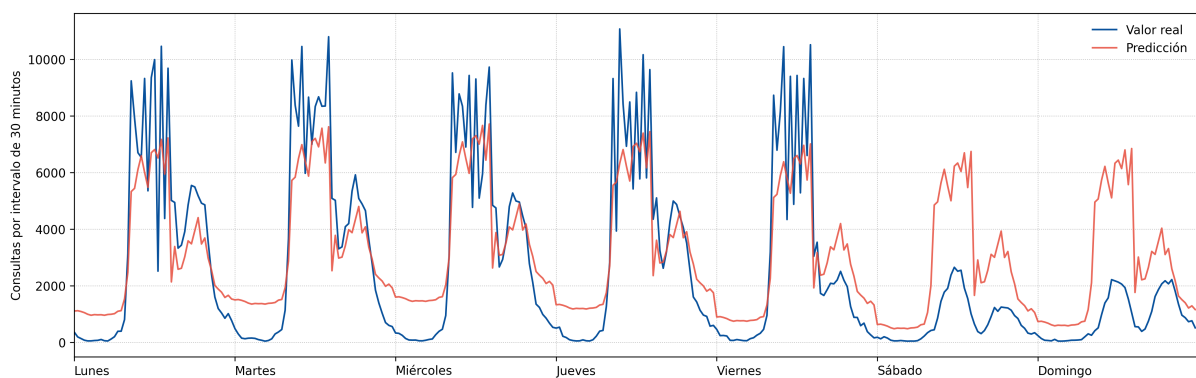


Figura 4.12: Valores reales y predicciones de SARIMAX.

En definitiva, los modelos clásicos como SARIMAX se quedan cortos frente a series con la variabilidad propia del tráfico web. Por un lado, se ha comprobado que tiene dificultad en capturar

varias estacionalidades de forma simultánea. Por otro lado, su naturaleza lineal le impide reproducir los cambios abruptos y la elevada varianza que caracterizan a este tipo de datos.

4.4.3 Modelo DeepAnT

El segundo modelo implementado es una adaptación de la arquitectura DeepAnT (Munir et al., 2019), explicado anteriormente en el marco teórico.

Arquitectura

La red utiliza la arquitectura propuesta por los autores originales. Concretamente, la red se compone de los siguientes elementos:

- Capa de entrada que recibe una ventana de W instantes y n características.
- Primer bloque convolucional: capa `Conv1D` de 32 filtros con *kernel* de tamaño 3, función de activación ReLU, seguida de una capa `MaxPooling1D` de tamaño 2.
- Segundo bloque convolucional idéntico al anterior.
- Capa `Flatten` que aplanar el mapa de características.
- Capa densa de una única neurona que predice el valor inmediatamente siguiente a la ventana.

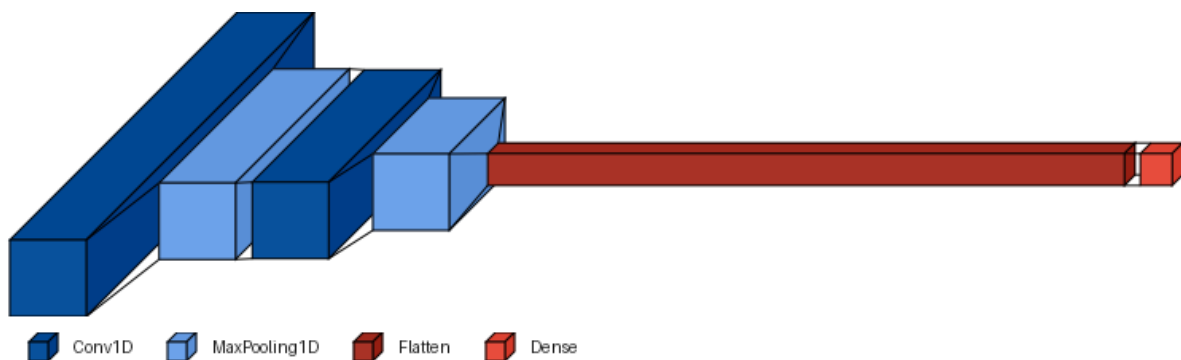


Figura 4.13: Arquitectura del modelo DeepAnT implementado.

Como función de pérdida se utilizó el error absoluto medio (MAE) y como optimizador el descenso por gradiente estocástico (SGD), siguiendo nuevamente la propuesta original del artículo.

Preparación de los datos

Una práctica muy habitual en el entrenamiento de redes neuronales es normalizar los datos (Aggarwal, 2023, sec. 2.7.2). Esta técnica transforma las variables de entrada de tal forma que compartan una escala similar. Con esto se consigue evitar que algunas variables tengan más importancias que otras por el mero hecho de tener una escala mayor. Adicionalmente, ayuda a algunos algoritmos de optimización a converger, como el descenso gradiente.

Las observaciones se normalizaron mediante un escalado `MinMaxScaler`, siguiendo la fórmula 4.6 (Aggarwal, 2023, pág. 64).

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (4.6)$$

La ecuación 4.6 se aplicó exclusivamente al conjunto de entrenamiento para evitar que el resto del *dataset* sesgara el escalado.

Posteriormente, se construyeron ventanas deslizantes de tamaño $W = 288$, equivalentes a 24 horas de historia a resolución de 5 minutos. Cada ventana actúa como entrada de la red y su correspondiente etiqueta es el valor del instante inmediatamente posterior.

A diferencia del modelo SARIMAX, en DeepAnT se incorporaron todas las variables exógenas disponibles: `festivo`, `dow_sin`, `dow_cos`, `time_sin` y `time_cos`. Estas variables son especialmente importantes en el caso de la red convolucional, ya que, a diferencia de la red LSTM, la convolucional no tiene contexto histórico y solo basa sus predicciones en los datos dentro de la ventana W .

Entrenamiento del modelo

Se entrenó el modelo durante 200 épocas, con una ventana histórica de 288 pasos que abarca un día completo. Para acelerar el entrenamiento, se utilizó *mixed precision* (NVIDIA Corporation, 2023), una técnica que reduce la precisión de los tensores en los cálculos intermedios de la red con el fin de aprovechar los *tensor cores* de la GPU ⁵.

Además, se optó por un `batch_size` de 256. Aunque la memoria del equipo físico admitía más tamaño de *batch*, se observó que afectaba perniciosamente a las predicciones.

La evolución de la pérdida durante el entrenamiento se muestra en la figura 4.14. La curva de entrenamiento desciende de forma monótona, mientras que la curva de validación decrece oscilando. Se aprecia que aunque la pérdida siga bajando, el error de validación se estabiliza en torno a la época 50. Esto da indicios de que a partir de la época 50, el modelo no va a mejorar sus pronósticos.

⁵Los núcleos *tensor core* están disponibles a partir de la arquitectura Volta de NVIDIA.

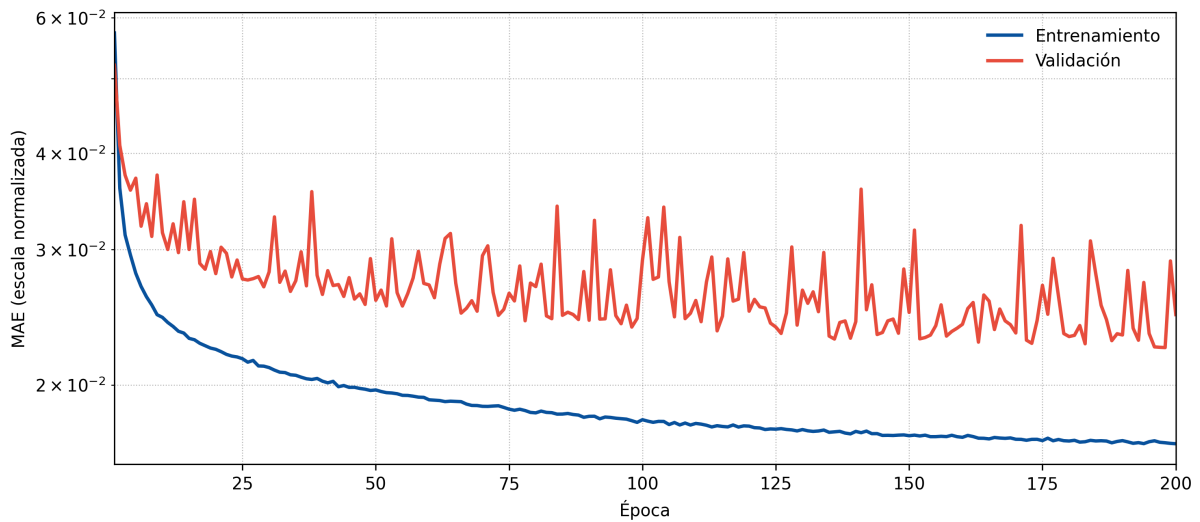


Figura 4.14: Evolución de los errores de entrenamiento y validación del entrenamiento de DeepAnT.

Calidad de las predicciones

Se evaluó la calidad de predicción del modelo sobre el conjunto de entrenamiento con anomalías y sin anomalías. Este segundo examen proporciona una medida más fiel del rendimiento puramente predictivo del modelo.

Métrica	Test completo	Test sin incidencias
MAE	96.53	82.45
RMSE	202.47	169.86
MAPE	0.785	0.758
sMAPE	0.842	0.680
Sesgo	5.95	21.52
R^2	0.828	0.881

Tabla 4.5: Métricas de predicción de DeepAnT sobre el conjunto de test.

En la tabla 4.5 se muestra que el modelo tiene un coeficiente de determinación del 0.88 al descartar las anomalías. Esta métrica baja cuando introducimos las interrupciones del sistema, como cabría esperar. El sesgo levemente positivo indica que el modelo tiende a subestimar el valor real ⁶.

Adicionalmente, en la figura 4.15 se representan los valores reales y los predichos sobre una semana representativa del conjunto de test.

⁶El motivo de este comportamiento es que el modelo fue entrenado con datos contaminados. Las caídas arrastran las predicciones hacia abajo.

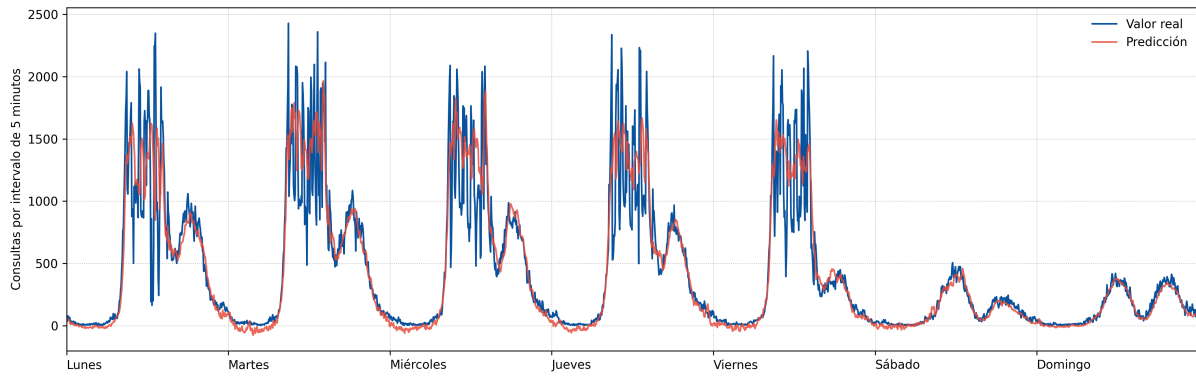


Figura 4.15: Valores reales y predicciones de DeepAnT.

Se observa que la red capta los patrones diarios y semanales del tráfico de peticiones. También se aprecia como efectivamente la red tiende a subestimar las observaciones reales.

4.4.4 Modelo LSTM

El tercer y último modelo predictivo implementado es una red recurrente de memoria a largo plazo, inspirada en el procedimiento propuesto por Malhotra et al. (2015).

Preparación de los datos

Al igual que con el modelo anterior, los datos se prepararon aplicando una normalización `MinMaxScaler` y se construyeron ventanas deslizantes del mismo tamaño $W = 288$, conservando también las mismas variables exógenas. La diferencia respecto a DeepAnT es que el procedimiento de Malhotra et al. (2015) exige entrenar la red con datos limpios de anomalías. En consecuencia, se desearon todas las ventanas contaminadas con valores atípicos⁷.

Arquitectura

La arquitectura empleada se compone de las siguientes capas:

- Capa de entrada que recibe una ventana de $W = 288$ instantes y n características.
- Primera capa LSTM con 64 unidades y retorno de la secuencia completa (`return_sequences=True`).
- Capa Dropout con probabilidad 0.2 para mitigar el sobreajuste.

⁷Si se descartasen las franjas de incidencias se producirían saltos abruptos en los datos. Por lo tanto, hay que descartar toda la ventana aunque se pierdan más datos.

- Capa `BatchNormalization` que estabiliza la distribución de las activaciones.
- Segunda capa LSTM con 32 unidades, sin retorno de la secuencia.
- Bloque `Dropout + BatchNormalization` análogo al anterior.
- Capa densa con $L = 10$ neuronas, encargada de predecir los próximos 10 pasos.



Figura 4.16: Arquitectura del modelo LSTM implementado.

Como función de pérdida se mantuvo el MAE; sin embargo, el optimizador se sustituyó por Adam, debido a su mejor desempeño durante las pruebas iniciales.

Predicción multi-paso

A diferencia de DeepAnT, que predice únicamente el siguiente instante, la red LSTM se entrena con un horizonte multi-paso de $L = 10$ valores. Este horizonte obligar al modelo a capturar dependencias temporales más extensas. No obstante, para el cómputo del *score* de anomalía únicamente se utiliza la predicción a un paso, manteniendo así la compatibilidad con el marco común de evaluación.

Esta decisión se aparta ligeramente del procedimiento original propuesto por Malhotra et al. (2015). El artículo emplea la distancia de Mahalanobis sobre el vector multidimensional de errores. Sin embargo, durante el desarrollo se observó que dicho enfoque conducía a peores resultados sobre este conjunto de datos, por lo que se acabó desechando.

Entrenamiento del modelo

En un inicio, se entrenó el modelo de forma similar a DeepAnT, utilizando 200 épocas. Sin embargo, al examinar las curvas de entrenamiento (4.17) se observó que en torno a la época 66 aumentaba el error de validación.

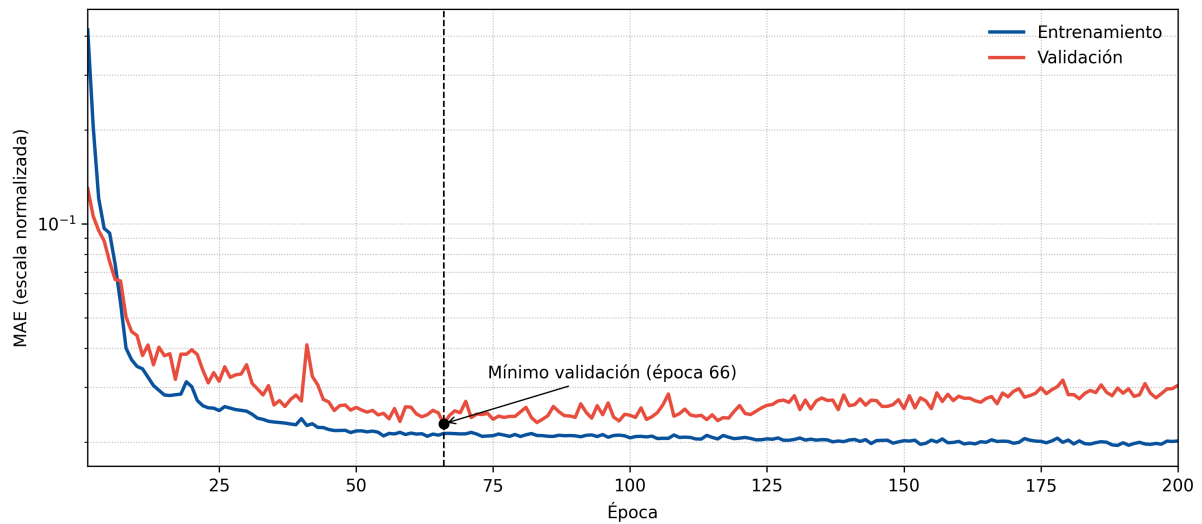


Figura 4.17: Curvas de entrenamiento de una ejecución preliminar de la LSTM con 200 épocas fijas.

Para mitigar este efecto, se utilizó la técnica de *early stopping* que para el entrenamiento si no ve mejoría. En concreto, se monitorizó la pérdida de validación con una paciencia de 10 épocas y, una vez disparada la parada, se restauraron los pesos correspondientes a la mejor época observada.

Tras incorporar la parada temprana, se relanzó el entrenamiento. Las curvas resultantes muestran en la figura 4.18.

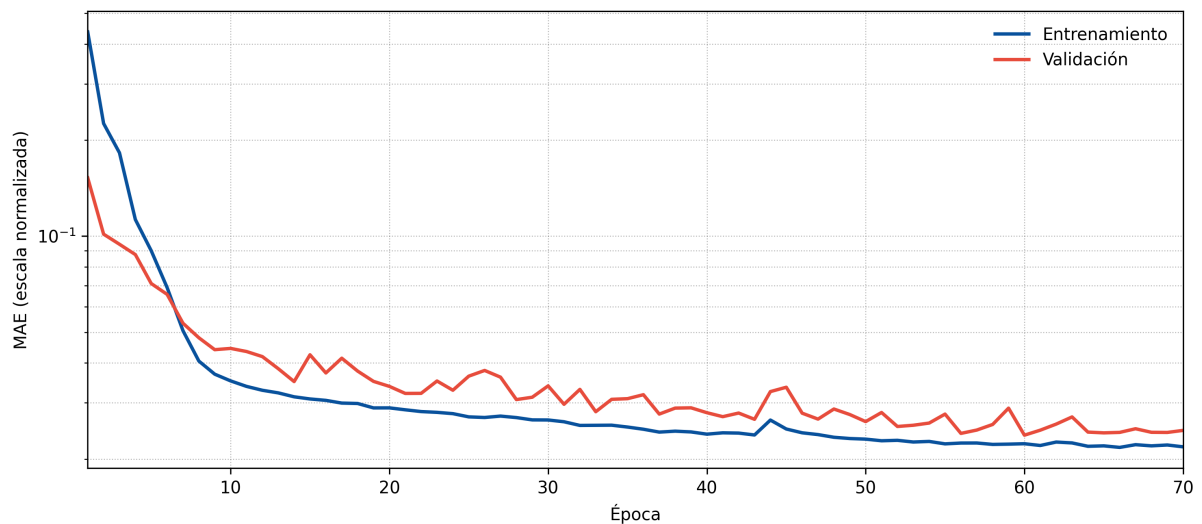


Figura 4.18: Curvas de entrenamiento de la LSTM con *early stopping* activado.

Calidad de las predicciones

Al igual que con DeepAnT, se calcularon las métricas de predicción de la red sobre el conjunto de test. Los resultados se muestran en la tabla 4.6.

Métrica	Test completo	Test sin incidencias
MAE	77.72	72.40
RMSE	176.26	160.39
MAPE	0.426	0.392
sMAPE	0.554	0.342
Sesgo	-12,08	-4,48
R^2	0.870	0.894

Tabla 4.6: Métricas de predicción de LSTM sobre el conjunto de test.

Se observa una ligera mejoría en todas las métricas de evaluación. El sesgo negativo indica que ahora la red tiende a sobreestimar; sin embargo, tiene menos sesgo que el modelo anterior.

La figura 4.19 muestra las predicciones de la red LSTM sobre una semana del conjunto de test.

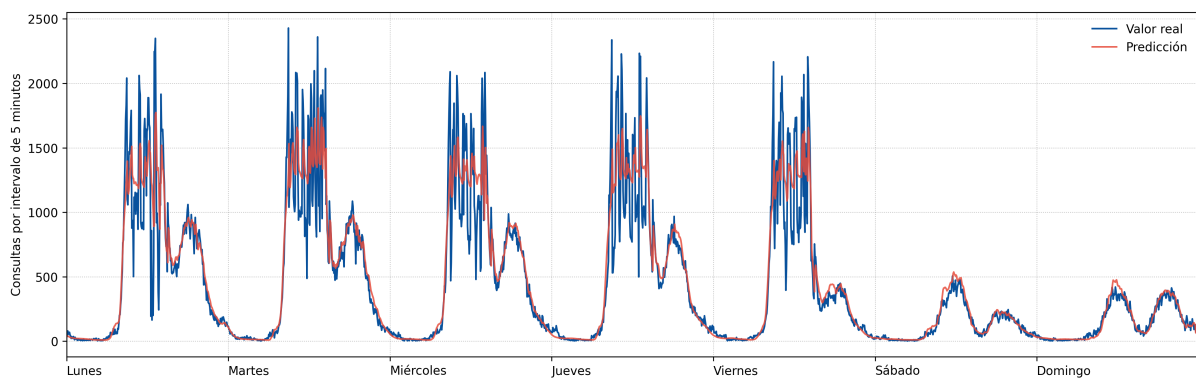


Figura 4.19: Valores reales y predicciones de la LSTM sobre una semana del conjunto de test.

Al igual que con DeepAnT, se observa que la red capta bien los patrones diario y semanal.

4.5 Ajuste de hiperparámetros

La elección de los hiperparámetros se basó en varios experimentos de entrenamiento. Estos experimentos se realizaron con la ayuda de la biblioteca Optuna, que automatiza la búsqueda y cambia los hiperparámetros de forma que maximicen el F_1 score de detección. Además, almacena los resultados en una base de datos SQLite para poder llevar un registro histórico de los modelos entrenados.

En la gráfica 4.20 se muestra la importancia que asocia Optuna a cada hiperparámetro. En el caso de DeepAnT, determinó que el tamaño de la ventana era el que más influía en los resultados.

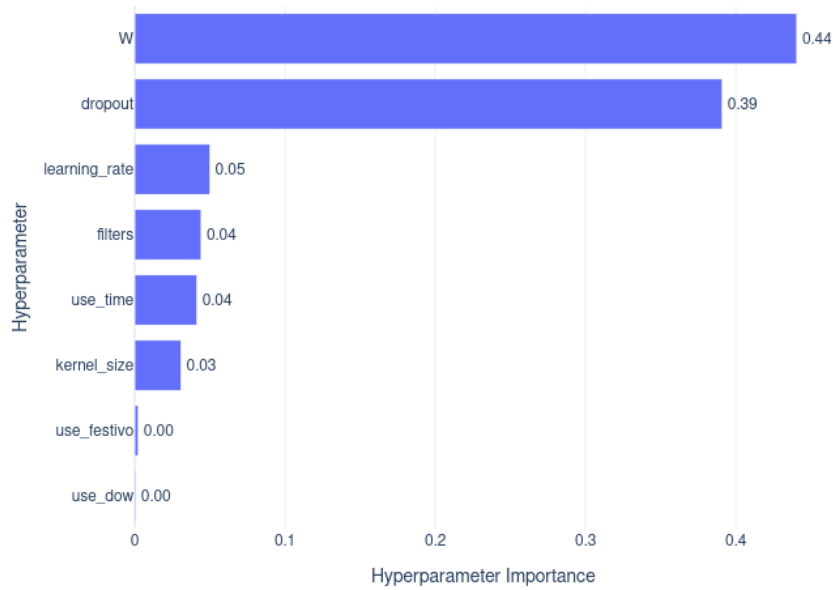


Figura 4.20: Gráfico de Optuna en el que se muestra la importancia de cada hiperparámetro.

El procedimiento se aplicó exclusivamente a los modelos DeepAnT y LSTM, debido a que SARIMAX ya cuenta con un mecanismo de búsqueda automatizado proporcionado por AutoARIMA.

4.6 Evaluación de los modelos

Una vez ajustados los modelos, se procedió a su evaluación comparativa sobre el conjunto de test. Para ello se utilizaron las dos métricas descritas en el marco común de evaluación: la métrica propia basada en eventos y la puntuación NAB. Los resultados se recogen en la tabla 4.7⁸.

Modelo	TP	FN	FP	Recall	F_1	NAB
Modelo base	116	142	64	0.450	0.530	15.03
SARIMAX	103	155	397	0.399	0.272	-78,01
DeepAnT	154	103	70	0.599	0.640	61.11
LSTM	166	91	69	0.646	0.675	65.53

Tabla 4.7: Métricas de detección de los modelos sobre el conjunto de test.

La LSTM se sitúa como el modelo con mejor desempeño en ambas métricas, seguida muy de cerca por DeepAnT. El modelo base, a pesar de su sencillez, supera ampliamente a SARIMAX, que cierra

⁸Las columnas TP, FN, FP, *recall* y F_1 se computan según la métrica propia basada en eventos descrita en el marco común de evaluación.

la clasificación con la peor F_1 y una puntuación NAB negativa. Esto último resulta especialmente llamativo, ya que indica que el comportamiento de SARIMAX es peor que el de un detector que nunca dispara alarmas.

Para mostrar el comportamiento de cada modelo, en la figura 4.21 se representan las alertas emitidas sobre un día con varias incidencias del conjunto de test. Cada modelo aparece con un marcador distinto.

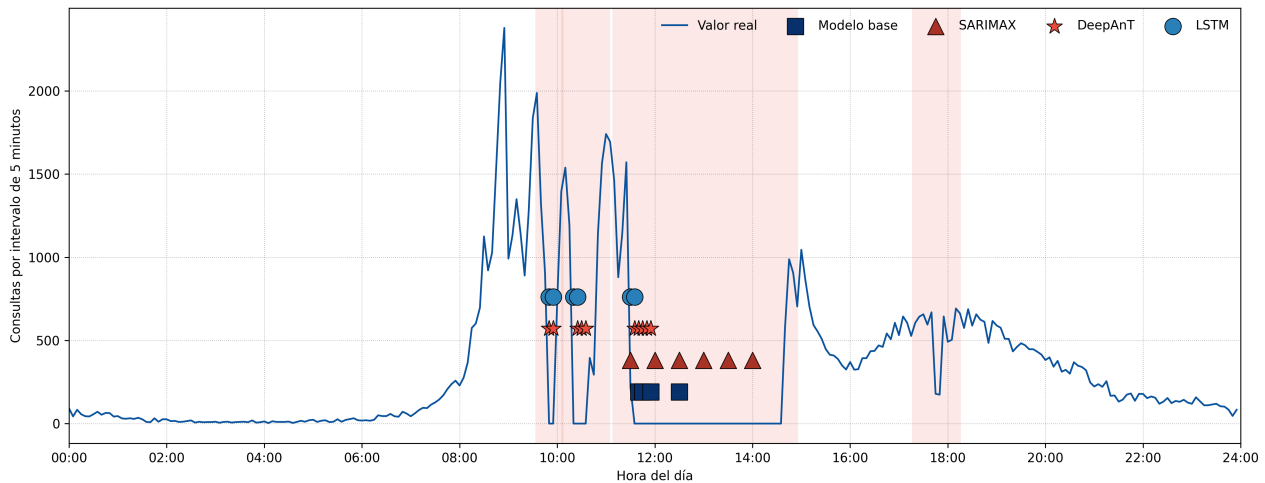


Figura 4.21: Detecciones de los cuatro modelos sobre un día con múltiples incidencias del conjunto de test.

Primero se observa que los únicos modelos que han detectado la primera incidencia fueron los basados en redes neuronales. Además, se ve que DeepAnT y SARIMAX son los modelos que más detecciones emiten durante las franjas anómalas. La LSTM dispara pocas alarmas pero todas dentro de las franjas reales. Ningún modelo es capaz de detectar la última incidencia de la tarde, lo que muestra las limitaciones del *anomaly score* y de los modelos basados en predicciones. Como la última franja ya de por sí tiene un tráfico bajo, una caída no llama tanto la atención como en horario laboral.

Para completar la comparativa desde el punto de vista del operador, en la tabla 4.8 se muestran dos magnitudes adicionales: la latencia mediana de detección⁹, que mide cuánto tarda cada modelo en lanzar la primera alarma de una incidencia real, y el tiempo total de falsa alarma, que cuantifica los minutos acumulados de aviso fuera de cualquier franja anómala.

⁹Se utiliza la mediana en lugar de la media ya que es más resiliente ante *outliers*. Por ejemplo, si hubiese un mantenimiento de 3 días y se detectase la caída al recuperar el tráfico habitual, el tiempo medio de detección se dispararía.

Modelo	Latencia mediana	Tiempo total de falsa alarma
Modelo base	32 min	13 h 10 min
SARIMAX	26 min	442 h 30 min
DeepAnT	21 min	11 h 25 min
LSTM	22 min	6 h 55 min

Tabla 4.8: Latencia mediana de detección y tiempo total de falsa alarma de los modelos sobre el conjunto de test.

DeepAnT y LSTM son los modelos más reactivos, con latencias medianas en torno a los 20 minutos. SARIMAX y el modelo base se sitúan algo por encima, sin llegar a comprometer el tiempo de reacción ante una incidencia real.

En cuanto al tiempo total de falsa alarma, la red LSTM es la opción más eficiente, con menos de siete horas acumuladas a lo largo del año y medio del conjunto de test. SARIMAX, por el contrario, acumula más de 442 horas de alarma falsa, equivalentes a unos 18 días de avisos. Esta cifra resulta inasumible para un operador y es motivo suficiente para descartar SARIMAX en el sistema de detección final.

4.6.1 Comparación con la literatura

A modo de referencia orientativa, los autores de NAB publican una tabla comparativa con los resultados de varios detectores sobre su *benchmark* oficial (Ahmad et al., 2019). La tabla 4.9 recoge dichas puntuaciones.

Detector	NAB <i>Standard Profile</i>
Detector perfecto	100.0
ARTime	74.9
Numenta HTM	70.5
CAD OSE	69.9
earthgecko Skyline	58.2
KNN CAD	58.0
Relative Entropy	54.6
Random Cut Forest	51.7
Twitter ADVec v1.0.0	47.1
Windowed Gaussian	39.6
Etsy Skyline	35.7
Bayesian Changepoint	17.7
EXPoSE	16.4
Detector aleatorio	11.0
Detector nulo	0.0

Tabla 4.9: Puntuación NAB de varios detectores de referencia sobre el *benchmark* oficial de Numenta (Ahmad et al., 2019).

A modo orientativo, los resultados obtenidos en este trabajo (DeepAnT: 61.11, LSTM: 65.53) se situarían en la mitad superior de la tabla, por encima de detectores como earthgecko Skyline o KNN CAD y por debajo de CAD OSE, Numenta HTM o ARTime.

No obstante, esta comparación debe tomarse con pinzas. Los modelos de la tabla se han evaluado sobre el corpus de NAB, un conjunto formado por 58 series temporales heterogéneas (métricas de AWS, tráfico de Twitter, datos de sensores, etc.) con características muy distintas al caso de uso de este trabajo. Aspectos como el período de las series, la naturaleza de las anomalías o la proporción entre datos limpios y anómalos influyen de forma decisiva en la puntuación. Por ello, estas cifras deben emplearse únicamente como referencia general y no como una comparativa estricta.

4.7 Aplicación de detección en tiempo real

Como cierre al desarrollo de este trabajo, se construyó una aplicación de monitorización en tiempo real que integra los detectores entrenados. La herramienta sustituye al sistema de alarmas rudimentario

empleado originalmente por la empresa. Se omitió el modelo SARIMAX debido a sus malos resultados de detección.

4.7.1 Utilidad y rol en la operación

El cuadro de mando se diseñó para reducir los falsos positivos del sistema anterior. El problema operativo no era la ausencia de alarmas, sino su exceso: cuando una herramienta dispara avisos con frecuencia, el operador termina ignorándolos y las caídas reales pasan desapercibidas.

La aplicación aborda este problema combinando los tres detectores mediante un criterio de consenso y centralizando en una única pantalla el valor instantáneo de la métrica, la estimación esperada, el estado de cada detector y un historial cronológico de las incidencias confirmadas.

4.7.2 Arquitectura y funcionamiento

La herramienta se implementó como aplicación web en Python con el *framework* Dash. De este modo, se reaprovecha parte del código de visualización del visor de incidencias y se expone la interfaz a través de un navegador convencional.

El sistema procesa cada observación nueva en cuatro etapas. En primer lugar, un módulo de ingesta proporciona los datos a un ritmo controlado; en el prototipo se reproduce el conjunto de test, mientras que en producción la fuente sería la telemetría real de la empresa. En segundo lugar, los tres modelos (modelo base, DeepAnT y LSTM) reciben la observación dentro de una ventana deslizante y devuelven su predicción y su dictamen individual. A continuación, un módulo de consenso combina los dictámenes: una franja solo se promueve a incidencia confirmada cuando al menos dos modelos coinciden en marcarla. Por último, el cuadro de mando actualiza la información visible al operador y registra la incidencia en el historial.

4.7.3 Descripción de la interfaz

La figura 4.22 muestra una captura del cuadro de mando durante una simulación. La interfaz se organiza en cuatro bloques.

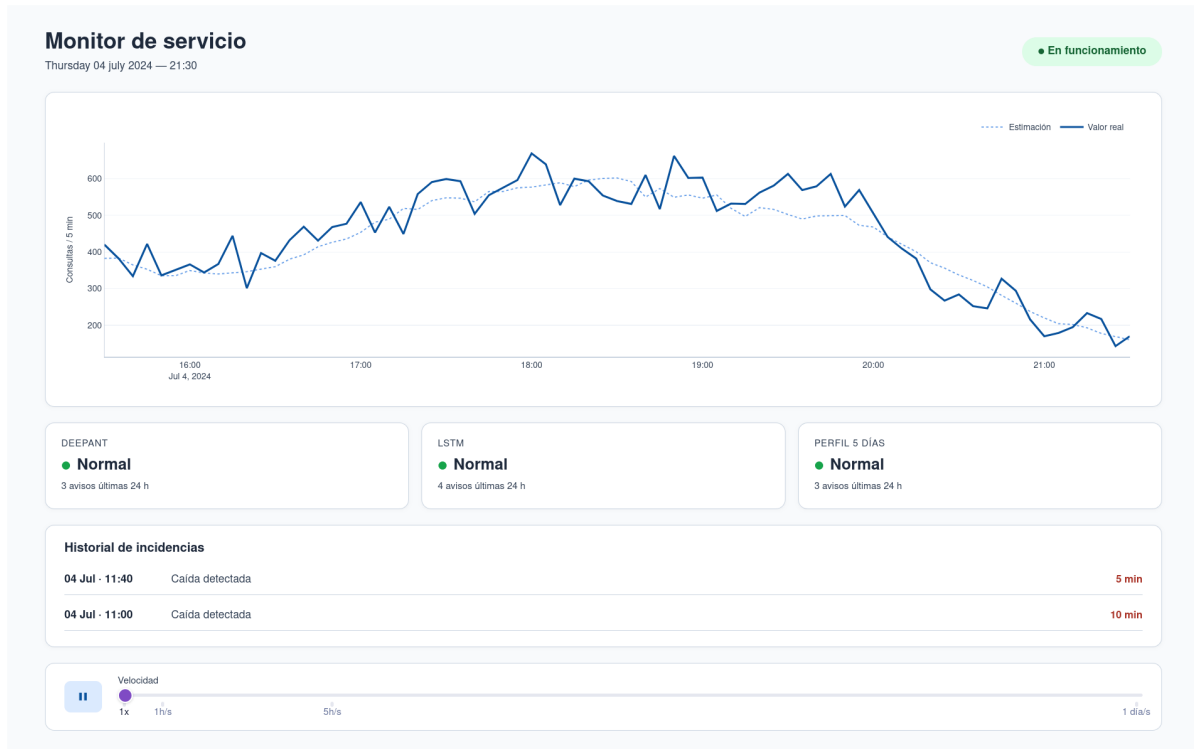


Figura 4.22: Cuadro de mando de la aplicación de detección en tiempo real.

En el bloque superior están el título del monitor, la marca temporal del instante simulado y una etiqueta de estado que pasa de verde («En funcionamiento») a un aviso destacado cuando se confirma una incidencia.

La gráfica principal representa el flujo de datos sobre una ventana móvil de las últimas horas. La línea continua corresponde al valor real y la discontinua a la estimación promedio de los tres modelos.

Bajo la gráfica se sitúan tres paneles individuales, uno por modelo (DeepAnT, LSTM y modelo base). Cada panel muestra el dictamen del detector (*Normal* o *Alerta*) y el número de avisos emitidos en las últimas veinticuatro horas. Mostrar los detectores por separado facilita el diagnóstico cuando un modelo concreto reporta más avisos de lo habitual.

El bloque «Historial de incidencias» recopila las caídas confirmadas más recientes en una tabla. Para cada incidencia se muestran la marca temporal de detección, una breve descripción y la latencia respecto al inicio estimado de la franja.

Por último, en la parte inferior se incluye un control de velocidad de simulación. Permite reproducir el conjunto de test a distintas velocidades y pausar la ejecución para inspeccionar episodios concretos. Este control no formaría parte del despliegue en producción.

Capítulo 5

Conclusiones

5.1 Conclusiones

Este trabajo ha desarrollado un sistema completo de detección de anomalías sobre series temporales de tráfico web, partiendo de un caso real de una empresa con un sistema de alarmas rudimentario. Se han abordado todas las fases planificadas, desde el preprocesamiento del conjunto de datos hasta la integración de los modelos en una aplicación de monitorización en tiempo real.

En cuanto a la comparativa de modelos, la red LSTM se ha situado como el mejor detector tanto en la métrica basada en eventos como en la puntuación NAB. DeepAnT obtiene resultados muy próximos y, además, requiere un tiempo de entrenamiento sensiblemente inferior. Sin tratarse de una comparativa estricta, ambos modelos se situarían en la mitad superior del *benchmark* oficial de Numenta, lo que sugiere que su rendimiento es razonable dentro del estado del arte. El modelo base, pese a su sencillez, ha demostrado ser una referencia sólida y útil para descartar implementaciones que no logren superar el rendimiento de una simple media histórica. SARIMAX, en cambio, ha resultado claramente insuficiente: su incapacidad para modelar dos estacionalidades simultáneas, unida a la naturaleza lineal de la familia ARIMA, lo deja muy por detrás del resto y obliga a descartarlo del sistema final.

Finalmente, la aplicación de detección a tiempo real integra los tres modelos seleccionados (modelo base, DeepAnT y LSTM) y los combina mediante un criterio de consenso, exigiendo que al menos dos modelos disparen simultáneamente para considerar una franja como incidencia. Esta estrategia aprovecha la complementariedad entre el enfoque estadístico y los basados en *deep learning* y reduce el número de falsos positivos individuales de cada detector.

5.2 Trabajo futuro

A lo largo del desarrollo se han identificado varias líneas de mejora que podrían continuar este trabajo en el futuro.

Bajar la latencia

Una vía de mejora directa consistiría en reducir la periodicidad de la serie de 5 minutos a 1 minuto, con el fin de acortar la latencia de detección. Durante el desarrollo se intentó esta aproximación, pero se descartó por dos razones. Por un lado, el coste de entrenamiento se disparó hasta hacer inviable la búsqueda de hiperparámetros. Por otro lado, los modelos entrenados con esta periodicidad produjeron predicciones de peor calidad, lo que en la práctica empeoró la latencia al retrasar el disparo de las alarmas. Queda abierto explorar arquitecturas que solventen este problema.

Modelos más punteros

Las arquitecturas empleadas en este trabajo (LSTM y CNN) son representativas del estado del arte previo a la irrupción de los *Transformers*, que constituyen el paradigma dominante en la actualidad. También se podrían explorar modelos híbridos como los discutidos en la sección de trabajos relacionados.

Mejores etiquetados

La calidad del etiquetado realizado en este trabajo está limitada por el grado de familiaridad con el sistema. Una posible mejora sería basar el etiquetado en el sistema de *ticketing* de incidencias de los usuarios. Otra opción sería etiquetar los datos junto a un equipo experimentado que conociese el sistema.

Referencias

- Aggarwal, C. C. (2017). *Outlier Analysis*. Springer. <https://doi.org/10.1007/978-3-319-47578-3>
- Aggarwal, C. C. (2023). *Neural Networks and Deep Learning: A Textbook* (2.^a ed.). Springer. <https://doi.org/10.1007/978-3-031-29642-0>
- Ahmad, S., Lavin, A., Purdy, S., & Agha, Z. (2017). Unsupervised real-time anomaly detection for streaming data [Online Real-Time Learning Strategies for Data Streams]. *Neurocomputing*, 262, 134-147. <https://doi.org/https://doi.org/10.1016/j.neucom.2017.04.070>
- Ahmad, S., Lavin, A., Purdy, S., Agha, Z., Danforth, I., Lewis, M., Scheinkman, L., Taylor, M., Gokhale, J., Silver, T., Smirnov, M., Marshall, A., Borgne, M. L., Takashina, Y., Kibish, S., ish-vlad, Baranski, C., Crowder, R., m.fab, ... breznak. (2019, diciembre). *numenta/NAB: v1.1* (Ver. v1.1). Zenodo. <https://doi.org/10.5281/zenodo.3571294>
- Box, G. E. P., Jenkins, G. M., & Reinsel, G. C. (2008). *Time Series Analysis: Forecasting and Control* (4.^a ed.). John Wiley & Sons.
- Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: identifying density-based local outliers. *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 93-104. <https://doi.org/10.1145/342009.335388>
- Brockwell, P. J., & Davis, R. A. (2016). *Introduction to Time Series and Forecasting* (3.^a ed.). Springer. <https://doi.org/10.1007/978-3-319-29854-2>
- Chang, I., Tiao, G. C., & Chen, C. (1988). Estimation of Time Series Parameters in the Presence of Outliers. *Technometrics*, 30(2), 193-204. <https://doi.org/10.1080/00401706.1988.10488367>
- Chen, C., & Liu, L.-M. (1993). Joint Estimation of Model Parameters and Outlier Effects in Time Series. *Journal of the American Statistical Association*, 88(421), 284-297. Consultado el 14 de junio de 2014, desde <http://www.jstor.org/stable/2290724>
- Fox, A. J. (1972). Outliers in Time Series. *Journal of the Royal Statistical Society. Series B (Methodological)*, 34(3), 350-363. Consultado el 1 de julio de 2014, desde <http://www.jstor.org/stable/2985071>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. The MIT Press.
- Hawkins, D. M. (1980). *Identification of Outliers*. Chapman & Hall. <https://doi.org/10.1007/978-94-015-3994-4>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.
- Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and Practice* (2nd). OTexts. <https://otexts.com/fpp2/>
- Kejariwal, A., Vallis, O., & Hochenbaum, J. (2015). *Introducing practical and robust anomaly detection in a time series*. Twitter Engineering. Consultado el 21 de abril de 2026, desde https://blog.x.com/engineering/en_us/a/2015/introducing-practical-and-robust-anomaly-detection-in-a-time-series

- Kim, T.-Y., & Cho, S.-B. (2018). Web traffic anomaly detection using C-LSTM neural networks. *Expert Systems with Applications*, 106, 66-76.
<https://doi.org/https://doi.org/10.1016/j.eswa.2018.04.004>
- Knorr, E. M., & Ng, R. T. (1998). Algorithms for Mining Distance-Based Outliers in Large Datasets. *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB)*, 392-403.
- Laptev, N., Amizadeh, S., & Flint, I. (2015). Generic and scalable framework for automated time-series anomaly detection. *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
<https://doi.org/10.1145/2783258.2788611>
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest [1550-4786/08 \$25.00 © 2008 IEEE]. *2008 Eighth IEEE International Conference on Data Mining*, 413-422.
<https://doi.org/10.1109/ICDM.2008.17>
- Malhotra, P., Vig, L., Shroff, G. M., & Agarwal, P. (2015). Long Short Term Memory Networks for Anomaly Detection in Time Series. *Proceedings of the European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*.
<https://api.semanticscholar.org/CorpusID:43680425>
- Munir, M., Siddiqui, S. A., Dengel, A., & Ahmed, S. (2019). DeepAnT: A Deep Learning Approach for Unsupervised Anomaly Detection in Time Series. *IEEE Access*, 7, 1991-2005.
<https://doi.org/10.1109/ACCESS.2018.2886457>
- NVIDIA Corporation. (2023). *Train With Mixed Precision* [Consultado en 2 de septiembre de 2026]. NVIDIA.
<https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html>
- Parametrix. (2024, julio). *CrowdStrike's Impact on the Fortune 500: An Impact Analysis* (Informe Técnico) (Disponible en: www.parametrixinsurance.com). Parametrix Solutions Inc. Consultado el 14 de febrero de 2026, desde https://cdn.prod.website-files.com/64b69422439318309c9f1e44/66a14199d3c5dde3b1c40aaa_CrowdStrikes%20Impact%20on%20the%20Fortune%20500_%202024%20_Parametrix%20Analysis.pdf
- Peña, D. (2010). *Análisis de series temporales*. Alianza Editorial.
- Ren, H., Xu, B., Wang, Y., Yi, C., Huang, C., Kou, X., Xing, T., Yang, M., Tong, J., & Zhang, Q. (2019). Time-Series Anomaly Detection Service at Microsoft. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 3009-3017.
<https://doi.org/10.1145/3292500.3330680>
- Shumway, R. H., & Stoffer, D. S. (2025). *Time Series Analysis and Its Applications: With R Examples* (5.^a ed.). Springer. <https://doi.org/10.1007/978-3-031-70584-7>
- Siarni-Namini, S., Tavakoli, N., & Siarni Namin, A. (2018). A Comparison of ARIMA and LSTM in Forecasting Time Series. *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 1394-1401. <https://doi.org/10.1109/ICMLA.2018.00227>
- Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley Publishing Company.
- Wong, J., Colburn, C., Meeks, E., & Vedaraman, S. (2015, febrero). *RAD: Outlier detection on big data*. Netflix TechBlog. Consultado el 21 de abril de 2026, desde <https://netflixtechblog.com/rad-outlier-detection-on-big-data-d6b0494371cc>
- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into Deep Learning* [<https://D2L.ai>]. Cambridge University Press.